



Implementierung eines Eye Tracking Systems in den Fahrsimulator der Hochschule Karlsruhe

Entwicklungsprojekt (Master)

am Institut für Energieeffiziente Mobilität
Fakultät für Maschinenbau und Mechatronik
Hochschule Karlsruhe - University of Applied Sciences

Lenard Haas,
Jonas Verleger

01.11.2024 - 10.06.2025

Erste(r) Gutachter(in): Patrick Rebling M.Sc.
Institute of Energy Efficient Mobility
Karlsruhe University of Applied Sciences

Zweite(r) Gutachter(in): Prof. Dr.-Ing. Philipp Nenninger
Institute of Energy Efficient Mobility
Karlsruhe University of Applied Sciences

© 2025 Lenard Haas, Jonas Verleger

Fassung: 9. Juni 2025



Dieses Werk ist unter einer Creative Commons Lizenz vom Typ Namensnennung zugänglich.

Um eine Kopie dieser Lizenz einzusehen, konsultieren Sie

<https://creativecommons.org/licenses/by/4.0/deed.de> oder wenden

Sie sich brieflich an Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit mit dem Titel

**Implementierung eines Eye Tracking Systems in den Fahrsimulator der Hochschule
Karlsruhe**

selbstständig angefertigt, alle benutzten Hilfsmittel vollständig und genau angegeben und alles einzeln kenntlich gemacht zu haben, was aus Arbeiten anderer unverändert oder mit Abänderungen entnommen wurde.

Karlsruhe, den 10. Juni 2025

Lenard Haas, Matrikelnummer 90272,

Jonas Verleger, Matrikelnummer 90922

Zusammenfassung

As part of this project, an eye tracker was implemented in a driving simulator at Karlsruhe University of Applied Sciences using inexpensive, commercially available hardware. The aim was to record the gaze position of test subjects in various traffic scenarios in order to analyse their perception and decision-making processes in road traffic more systematically. The aim of the project was not to develop a completely new tracking system from scratch. Instead, existing eye tracking systems were to be evaluated for their applicability in the university's driving simulator and implemented. The main part of the project consisted of finding a suitable eye tracking system, implementing it in the CARLA environment and finding solutions to any technical problems that might arise.

After a market analysis, the Beam Eye Tracker from Eyeware Tech was selected due to its user-friendly API, low purchase price, low hardware requirements and accuracy that met the requirements of the application. The implementation was carried out using Python with the Beam SDK. The tracking data was linked to CARLA, in particular to the data from the Instance Segmentation Sensor, in order to infer the objects viewed in the simulation from the gaze position estimated by the eye tracker. This allows the eye tracker not only to output the test person's gaze position in x and y coordinates relative to the screen, but also to recognise which object the test person is looking at within the CARLA simulation environment at any given time. The interface between the eye tracker data and the CARLA Instance Segmentation Sensor data was tested in a specially created test environment in CARLA.

A key technical problem was the curved projection screen of the driving simulator, which led to distortions in the recorded gaze data. To compensate for this, a software-based correction was implemented that uses mathematical functions to minimise the error caused by the curved screen. The accuracy of the implemented tracking system is sufficient to be able to determine which object the test person is looking at at any given time for larger objects in the CARLA simulation world. These are good results, especially considering the low financial outlay compared to eye tracking systems that cost several hundred euros. It was possible to implement the eye tracking system free of charge. The only costs incurred were those of the webcam used. Implementation in the driving simulator and in the CARLA environment was completed in full.

The work proves that even with inexpensive hardware and software, it is possible to functionally integrate eye tracking systems into driving simulators with large, curved screens, thus laying the foundation for future experiments in driving behaviour analysis. The work also identifies potential approaches for further improving tracking accuracy in the future.

Abstract

Im Rahmen dieser Projektarbeit wurde ein Eye Tracker mithilfe kostengünstiger und handelsüblicher Hardware in einen Fahrsimulator der Hochschule Karlsruhe implementiert. Ziel war es, die Blickposition von Testpersonen in verschiedenen Verkehrsszenarien zu erfassen, um deren Wahrnehmungs- und Entscheidungsprozesse im Straßenverkehr systematischer analysieren zu können. Es war nicht das Ziel der Arbeit, ein eigenes Tracking System vollständig neu zu entwickeln. Stattdessen sollten bereits existierende Eye Tracking Systeme auf ihre Anwendbarkeit im Fahrsimulator der Hochschule bewertet und implementiert werden. Der Hauptbestandteil der Arbeit bestand darin, ein geeignetes Eye Tracking System zu finden, dieses in die CARLA Umgebung zu implementieren und Lösungen für gegebenenfalls vorhandene technische Probleme zu finden.

Nach einer Marktanalyse wurde der Beam Eye Tracker von Eyeware Tech aufgrund seiner benutzerfreundlichen API, kostengünstigen Anschaffung, geringen Hardwareanforderungen und einer Genauigkeit, die den Anforderungen der Anwendung genügte, ausgewählt. Die Implementierung erfolgte mithilfe von Python unter Verwendung der Beam SDK. Die Trackingdaten wurden mit CARLA verknüpft, insbesondere mit den Daten des Instance Segmentation Sensors, um von der vom Eye Tracker geschätzten Blickposition auf die in der Simulation betrachteten Objekte zu schließen. Dadurch kann der Eye Tracker nicht nur die Blickposition der Testperson in x- und y-Koordinaten relativ zum Bildschirm ausgeben, sondern erkennt auch, auf welches Objekt die Testperson innerhalb der CARLA Simulationsumgebung zu jedem Zeitpunkt schaut. Innerhalb einer eigens dafür erstellten Testumgebung in CARLA konnte die Schnittstelle zwischen den Daten des Eye Trackers und den Daten des CARLA Instance Segmentation Sensors getestet werden.

Ein zentrales technisches Problem stellte die gekrümmte Projektionsleinwand des Fahrsimulators dar, die zu Verzerrungen der erfassten Blickdaten führte. Zur Kompensation wurde eine softwarebasierte Korrektur implementiert, die eine mathematische Funktionen verwendet, um den Fehler durch die gekrümmte Leinwand weitgehend zu minimieren. Die Genauigkeit des implementierten Tracking Systems reicht aus, um bei größeren Objekten in der CARLA Simulationswelt Aussagen darüber treffen zu können, auf welches Objekt die Testperson zum jeweiligen Zeitpunkt ihren Blick richtet. Besonders unter dem Aspekt des geringen finanziellen Aufwands sind das relativ zu Eye Tracking Systemen, die mehrere hundert Euro kosten, gute Ergebnisse. Es war möglich, das Eye Tracking System kostenfrei zu implementieren. Die einzigen Kosten, welche entstanden sind, waren die der verwendeten Webcam. Eine Implementierung in den Fahrsimulator und in die CARLA Umgebung wurde vollständig durchgeführt.

Die Arbeit belegt, dass bereits mit kostengünstiger Hard- und Software eine funktionale Integration von Eye Tracking Systemen in Fahrsimulatoren mit großen und gekrümmten Leinwänden möglich ist und legt damit die Grundlage für zukünftige Experimente zur Fahrverhaltensanalyse. Die Arbeit identifiziert zudem potentielle Ansätze, mit denen die Tracking Genauigkeit in Zukunft weiter verbessert werden kann.

Inhaltsverzeichnis

Zusammenfassung	v
Abstract	vii
Inhaltsverzeichnis	x
1 Einführung	1
1.1 Zielsetzung	2
1.2 Lösungsansatz	2
2 Eye Tracker	3
2.1 Grundlegende Hardwarevoraussetzungen	3
2.2 Marktanalyse	4
2.3 Beam Eye Tracker	6
3 Implementierung Eye Tracker	7
3.1 Einrichtung im Fahrsimulator	7
3.2 Beam Eye Tracker SDK	10
3.3 Schnittstelle zu CARLA	11
4 CARLA Umgebung	13
4.1 Aufbau Testszenario in CARLA	13
4.2 Sensor Auswertung	16
4.3 Darstellung der Daten in CARLA	19
5 Korrektur gekrümmte Leinwand	21
5.1 Problemursache	21
5.2 Lösungsansatz	23
6 Auswertung	33
7 Fazit	37
7.1 Auswertung und Zusammenfassung	37
7.2 Ausblick	38
Appendix	
A Code des Eye Trackers	39
B Matlab Skripte & Messwerte	41

C Demo Video Eye Tracker	43
Abbildungsverzeichnis	46
Abkürzungsverzeichnis	47
Symbolverzeichnis	51
Literatur	53

1 Einführung

Das menschliche Fahrverhalten ist bedingt durch die Vielzahl der möglichen Situationen im Straßenverkehr vielseitig. Um das Verhalten in verschiedenen Situationen zu untersuchen, müssen entweder Daten aus dem realen Verkehr ausgewertet werden oder bestimmte Situationen in einer kontrollierten Testumgebung nachgestellt werden. In einer realen Testumgebung ist es allerdings schwer, bestimmte Situationen gezielt nachzustellen. Die Umgebungsbedingungen des realen Verkehrs mit unterschiedlichen Fahrumgebungen wie Landstraßen, Autobahnen, Städten oder verschiedene Verkehrsteilnehmern wie Autofahrern, Straßenbahnen, Radfahrern und Fußgängern sind komplex. Ein anderer Ansatz ist es, die Umgebung zu simulieren. Auch für die Nachbildung von gefährlichen Situationen im Straßenverkehr weist eine Simulation aus finanzieller und praktischer Sicht Vorteile auf. Verschiedene Verkehrssituationen können mit kleineren finanziellen Aufwänden wiederholbar und zeitlich flexibel durchgeführt werden. Das Automotive Mobility Lab des Instituts für energieeffiziente Mobilität (IEEM) entwickelt aus diesem Grund ein Simulationsframework, welches mit unterschiedlichen Fortbewegungsmethoden (Auto, Fahrrad, Gehen) und in unterschiedlicher Realitätsnähe genutzt werden kann. Dadurch sind Human in the Loop-Fahrversuche möglich, bei denen Fahrversuche mit einer realen Testperson in einer simulierten Umgebung durchgeführt werden können.

Als Softwarebasis dient das Open-Source-Projekt CARLA. Dieses ermöglicht es, verschiedene Verkehrsszenarien nachzustellen. Die Analyse des menschlichen Verhaltens im Straßenverkehr kann somit ohne Gefahren und in vielfältigen simulierten Szenarien erfolgen. Die Server-Client Struktur von CARLA lässt ebenso mehrere Probanden in demselben Verkehrsszenario zu. So können menschliche Interaktionen realitätsnah nachgebildet werden.

Um die Ursachen bestimmter Entscheidungen im Straßenverkehr zu ergründen, soll in dem Fahrsimulator ein Eye Tracking System integriert werden. Durch die Auswertung der Blickposition des Fahrers sollen Informationen zum Wahrnehmungs- und Entscheidungsprozess des Fahrers in gewissen Situationen gewonnen werden. Hierdurch soll deutlich werden, was die Probanden während der Testfahrt sehen, worauf diese sich konzentrieren und welche Handlungsoptionen sie in Gefahrensituationen abwägen.

1.1 Zielsetzung

Im Rahmen dieser Projektarbeit soll ein kamera basierter Eye Tracker in den Fahrsimulator der Hochschule bzw. in die CARLA Simulationsumgebung integriert werden.

Dazu sollen verschiedene verfügbare Eye Tracker auf die Anwendbarkeit und Integrierbarkeit in CARLA bewertet werden. Zur Erprobung des Eye Trackers soll ein Test-szenario in CARLA erstellt werden. Weitere Vorgaben sind die Projektion des Blickpunktes auf die Leinwand und eine Ausgabe der aufgezeichneten Daten. Außerdem sollen die Daten innerhalb der Simulationsumgebung mit den Daten des Eye Trackers vereint werden. Dafür soll vor allem der Instance Segmentation Sensor innerhalb von CARLA zum Einsatz kommen. Dieser wird in einem späteren Abschnitt genauer erklärt. Ziel wird es sein, die Daten des Eye Trackers mit den Daten dieses Sensors zu vereinen, um Aussagen über die von der Testperson angeschauten Objekte in der Simulation zu erhalten.

1.2 Lösungsansatz

Zur Zielerreichung müssen zuerst die Grundlagen von Eye Tracker Systemen und der CARLA-Simulationsumgebung erarbeitet werden. Für die Integration in das Gesamtsystem soll ein objektorientierter Python Code entwickelt werden. Damit die Ziele erreicht werden können, ist vor allem eine intensive Auseinandersetzung mit den Application Programming Interfaces (API)s der verschiedenen beteiligten Systemen notwendig. Eine Eigenentwicklung eines Eye Tracking Systems ist technisch und mathematisch anspruchsvoll und würde den zeitlichen Rahmen einer Projektarbeit übersteigen. Aus diesem Grund muss ein bereits vorhandenes System gefunden werden, welches kostengünstig ist und welches mindestens eine API bietet, mit welcher die ausgegebenen Daten weiterverarbeitet werden können. Außerdem muss sichergestellt werden, dass die Krümmung der Leinwand bei der Verwendung des Eye Tracking Systems nicht zu Ungenauigkeiten führt. Entweder wird ein Eye Tracking System eingesetzt, welches gekrümmte Leinwände nativ unterstützt oder es muss eine selbst entwickelte Lösung erarbeitet werden.

2 Eye Tracker

2.1 Grundlegende Hardwarevoraussetzungen

In diesem Kapitel wird ein grundlegender Einblick in die Hardwarevoraussetzungen für ein Eye Tracking System bereitgestellt. Daraufhin wird eine Marktanalyse durchgeführt, um verfügbare Eye Tracker auf deren Anwendbarkeit, Integration in CARLA und Kosten zu untersuchen.

Eye Tracking Systeme bestimmen die Mitte der Pupille, leiten die Rotation des Augapfels ab und bestimmen dann mithilfe von mathematischen Algorithmen die Blickposition [1]. Dabei wird zwischen remote Systemen und Head Mounted Systemen unterschieden. Bei den remote Systemen sitzt das Eye Tracking System nicht auf dem Kopf der Testperson. Es ist an einer von der Testperson entfernten Stelle positioniert und erfasst von dort aus die Blickposition. Häufig werden solche remote Systeme verwendet, wenn die Blickposition der Testperson auf einen Bildschirm untersucht werden soll. Dadurch ist die Erfassung meist auf den jeweiligen Bildschirm beschränkt. Die Erfassung von Objekten in der realen dreidimensionalen Welt, welche über den Bildschirm hinausgeht, ist in der Regel nicht möglich. Es gibt allerdings ein paar Anbieter, die remote Systeme anbieten, welche auch Blickpositionen in der realen dreidimensionalen Umgebung erfassen. Bei den tragbaren Eye Tracking Systemen hingegen wird der Testperson das System aufgesetzt. Solche Systeme erfassen die Augenbewegung dadurch aus nächster Nähe. Ein Head Tracking System eignet sich insbesondere für Tracking-Experimente in der realen Welt. [1],[2]

Die minimalen Hardwarevoraussetzungen für einen Eye Tracker belaufen sich auf eine Kamera, eine Lichtquelle und einen Computer. In teureren Systemen werden teilweise zwei Kameras eingesetzt. Dadurch kann sich die Tracking-Qualität verbessern, diese Systeme sind meist aber auch komplexer [1]. Zudem wird hier auch Software benötigt, welche mehrere Kameras gleichzeitig auswerten kann. In diesem Projekt soll der Blick auf eine Leinwand erfasst werden. Aus diesem Grund reicht ein remote System aus. Es wird eine einzelne Kamera verwendet, da die Auswahl der in Frage kommenden Softwarelösungen sonst zu sehr eingeschränkt wird. Unabhängig vom eingesetzten System gilt, dass die Genauigkeit gesteigert werden kann, indem eine hochauflösende Kamera genutzt wird [2]. Ergebnisse hängen allerdings auch stark mit dem Abstand der Testperson zur Kamera ab. Sitzt die Testperson in geringem Abstand zu der Kamera, dann ist auch eine kleinere Auflösung ausreichend. In unseren Experimenten wurde eine Kamera mit einer Auflösung von 1920x1080 Pixel eingesetzt. Diese Auflösung hat sich in Kombination mit einem Abstand von unter einem Meter zur Kamera als ausreichend erwiesen, sodass der Eye Tracker die Augenbewegung erfassen

konnte. Auch Kameras mit höherer Bildrate können bessere Ergebnisse liefern. Allerdings reicht eine Bildwiederholrate von 30 Bildern pro Sekunde für die Erfassung von Augenbewegungen in vielen Anwendungen aus [3]. Kameras haben im Vergleich zum menschlichen Auge die Fähigkeit, infrarotes (IR) Licht zu erfassen. Digitalkameras und Handykameras können je nach eingesetzter Sensortechnologie bereits IR-Strahlung wahrnehmen. Oft sind allerdings Filter eingebaut, die IR-Strahlung größtenteils herausfiltern, da die Kameras die menschliche Sicht abbilden sollen [1]. Es gibt allerdings spezielle IR-Kameras, deren Sensor auf einer anderen Technologie basiert, sodass diese besonders im IR-Bereich sensitiv sind. Einsatzgebiete sind bspw. Wärmebildkameras oder Nachtsichtkameras. Im Kontext von Eye Tracking Systemen können solche Kameras auch bei schlechten Lichtverhältnissen, im Vergleich zu einer Kamera, welche nur im menschlich sichtbaren Frequenzbereich operiert, noch verwertbare Bildinformationen erhalten [4]. Durch das Anbringen von Infrarotlichtquellen und dem Einsatz einer solchen Infrarotkamera können auch bei schlechteren Lichtverhältnissen verlässliche Ergebnisse erzielt werden [5].

Auf die genaue Funktionsweise von Eye Tracking Systemen und den dafür eingesetzten mathematischen Algorithmen soll in dieser Projektarbeit nicht eingegangen werden, da es nicht die Zielsetzung ist, ein solches System selbst zu entwickeln. Für die vorliegende Arbeit ist lediglich relevant, dass die Ergebnisqualität des Eye Tracking System stark von der Art des eingesetzten Systems, den Lichtverhältnissen und dem Abstand der Testperson zur Kamera abhängig ist.

2.2 Marktanalyse

Wie oben bereits beschrieben wird eine Marktanalyse von bereits verfügbaren Eye Trackern durchgeführt. Die Kriterien, die das System erfüllen soll, wurden bereits in vorherigen Abschnitten erläutert.

Ein erster Softwarekandidat ist ein Open Source Tracking Projekt auf GitHub [6]. Die Ausgabe dieses Trackers ist in Abbildung 1 zu erkennen. Das System orientiert sich hauptsächlich an den Augen und der Nase der Testperson. Dadurch kann es bestimmen, in welche Richtung der Kopf einer betrachteten Person geneigt ist. Zudem kann die Software das Blinzeln der Augen erkennen.

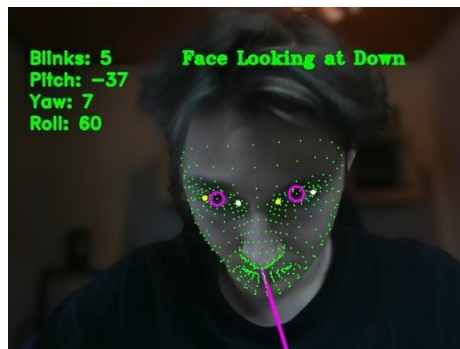


Abbildung 1: Ausgabe des Face-Gaze-Trackers, Quelle: [6]

Die Software schätzt nicht die Blickposition der Testperson auf dem Bildschirm, sondern gibt nur an, in welcher Lage sich der Kopf befindet. Diese Daten könnten aber als Grundlage für eine Weiterentwicklung dienen. Da allerdings nur die Ausrichtung des Gesichts und nicht der Pupillen in die Ausgabe einfließt, werden keine genauen Ergebnisse durch diese Herangehensweise erwartet.

Eine weitere Option stellt das open source Projekt 'Eyegaze Tracker' dar. Diese Software erfasst nicht die Gesichtsausrichtung, sondern die Pupillen. Dadurch kann sie eine qualitative Aussage über die Blickrichtung treffen. So wie in Abbildung 2 zu erkennen ist, kann bspw. die Aussage getroffen werden, dass die Testperson nach links schaut. Auch hier wird keine Blickposition auf dem Bildschirm geschätzt. Allerdings werden die Pupillen erfasst und die grobe Blickrichtung bestimmt. Die Blickposition kann gegebenenfalls mit den vorhandenen Daten durch eine Weiterentwicklung des vorhandenen open source Codes bestimmt werden. Dies würde allerdings einen erheblichen Entwicklungsaufwand bedeuten.

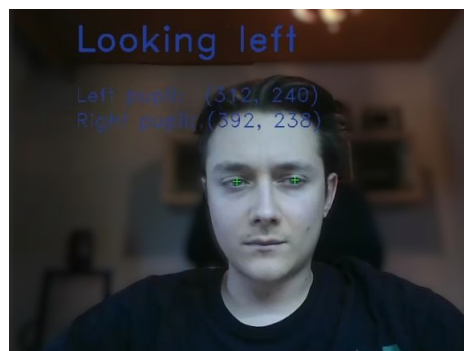


Abbildung 2: Ausgabe des Eyegaze-Trackers, Quelle: [7]

Neben den oben genannten Projekten wurden auch noch einige weitere auf ihre Einsetzbarkeit in diesem Projekt untersucht. Diese hatten größtenteils ähnliche Eigenschaften und Einschränkungen. Eye Tracker die einen größeren Funktionsumfang bieten, wie beispielsweise das System 'Tobii Eye Tracker' haben den Nachteil, dass sie einen, je nach Budget, hohen finanziellen Aufwand bedeuten. Es wurde sich auf Grundlage umfangreicher Recherche und experimenteller Erprobung für den Beam Eye Tracker von Eyeware Tech entschieden. Dieser hat die in dieser Arbeit definierten Anforderungen erfüllt und soll als Basis für die weitere Entwicklung dienen.

2.3 Beam Eye Tracker

Der Beam Eye Tracker ist für Gaming-Anwendungen und dem Einsatz von einfacher Hardware wie normalen Webcams entwickelt. Neben der Schätzung der Blickposition auf einen Punkt auf dem Bildschirm verfügt das Programm über eine Kopfpositionserkennung. Durch die Funktion 'Gaming Erweiterungen' teilt der Beam Eye Tracker die Trackingdaten über seine Application Programming Interface (API) mit anderen Programmen.

Die Beam API kann über die Programmiersprachen C++, C, C# und Python angesprochen werden [8]. Im Rahmen dieser Projektarbeit wird die API mit Python angesprochen. Verschiedene Klassen in der API ermöglichen eine individuelle Einbindung der Eye Tracking Daten in eine selbst entwickelte Anwendung. Relevant für diese Projektarbeit ist die Klasse „screen_gaze_info“. Diese beinhaltet die x- und y- Koordinaten und die Angaben, ob die Augen erkannt werden und wie gut die Schätzung ist.

Durch die anwenderfreundliche API des Beam Software Development Kit (SDK) und den niedrigen Preis von 29,99€ des Eye Trackers, fiel die Entscheidung, diesen für das Projekt zu nutzen. In Rücksprache mit den Beam-Entwicklern wurde der Beam Eye Tracker für dieses Projekt unentgeltlich zur Verfügung gestellt.

3 Implementierung Eye Tracker

3.1 Einrichtung im Fahrsimulator

Eyeware Tech hat ihren Beam Eye Tracker so aufgebaut, dass er mit handelsüblicher Hardware verwendet werden kann. Das System kann bereits mit einer handelsüblichen Universal Serial Bus (USB) Webcam oder mit einer an den Computer angeschlossenen Smartphone-Kamera eingesetzt werden. Damit die Integration des Eye Trackers in den Fahrsimulator gelingt, müssen mehrere Voraussetzungen geschaffen werden. Diese werden in den folgenden Abschnitten beschrieben.

Die Positionierung der Kamera ist von zentraler Bedeutung. Diese soll laut Eyeware Tech mittig zum Bildschirm ausgerichtet sein [9]. Die Sitzposition der Testperson hingegen muss nicht mittig zur Kamera sein. Die Testperson kann auch relativ zur Kamera links oder rechts sitzen. Diese Annahme konnte durch selbst durchgeführte Tests bestätigt werden. In den Eye Tracker Einstellungen müssen auch die Kameraposition und der Neigungswinkel korrekt eingestellt werden. Die genauesten Ergebnisse konnten erzielt werden, als die Kamera im Fahrsimulator mittig zur Leinwand in der Mitte des Armaturenbretts angebracht war. Die Kameraposition wurde im Eye Tracker als 'unten' und mit einem Neigungswinkel von 0° angegeben.

Grundlage für verlässliche Ergebnisse ist es, die Kalibrierung, welche der Beam Eye Tracker integriert hat, vor jeder Sitzung im Fahrsimulator durchzuführen. Insbesondere wenn mehrere Personen nacheinander fahren, die sich von ihrer Körpergröße stark unterscheiden. Die in den Beam Eye Tracker integrierte Kalibrierung besteht aus fünf Punkten, die nacheinander auf der Leinwand erscheinen. Vier Punkte werden dabei in die vier Ecken der Leinwand positioniert. Der fünfte Punkt befindet sich in der Mitte der Leinwand. Während der Kalibrierung blickt die Testperson nacheinander auf diese Punkte. Sofern die Testperson einen Punkt fest fokussiert hat, klickt sie mit der Maus auf diesen Punkt. Der Eye Tracker verknüpft dann die aktuelle Blickposition mit den Koordinaten des Punktes auf der Leinwand. Nach diesem Prinzip kalibriert sich der Eye Tracker selbst und verknüpft die Position der Pupillen mit einer Position auf der Leinwand. Wie dieser Kalibrierungsprozess im Detail funktioniert, ist nicht bekannt.

Für die Tests des Eye Trackers wurde der mittelgroße Fahrsimulator in der Hochschule Karlsruhe (HKA) eingesetzt. Da die Leinwand sehr groß ist und das Kalibrierungsfenster des Eye Trackers nicht verkleinert werden kann, befinden sich die Kalibrierungspunkte teilweise so weit in den Ecken, dass sie vor allem unten nicht mehr auf der Leinwand angezeigt werden. Das erschwert eine präzise Interaktion. Außerdem sollten während der Kalibrierung keine großen Kopfbewegungen gemacht werden. Experimente zeigten, dass das die Tracking-Genauigkeit verringern kann. Da die Punkte

durch die breite Leinwand aber außerhalb des Sichtfelds der Testperson liegen, muss diese zwangsweise eine Kopfbewegung machen.

Eine Herangehensweise, um diese Probleme zu minimieren, ist es, die Kalibrierung immer zu zweit durchzuführen. Die Testperson ist dafür verantwortlich, auf die Kalibrierungspunkte zu schauen und die zweite Person klickt auf diese mit der Maus. Da die Punkte teilweise außerhalb des Sichtfeldes liegen und die Testperson den Kopf nicht zu sehr bewegen sollte, ist es am besten, wenn diese nicht direkt auf den jeweiligen Punkt schaut, sondern leicht daneben. Wenn der Punkt bspw. unten rechts im Eck erscheint, dann richtet die Testperson bei der Kalibrierung ihren Blick leicht versetzt nach links. Zum einen wird dadurch die Kalibrierung insgesamt verbessert, da der Kopf nicht bewegt werden muss. Zusätzlich wird dadurch ein Fehler leicht abgeschwächt, welcher durch die Krümmung der Leinwand entsteht.

Zwischenzeitlich wurde im Rahmen dieser Projektarbeit auch der Ansatz einer eigenen Kalibrierung verfolgt. Diese hat vorausgesetzt, dass der Eye Tracker bereits einmal sehr grob mit dem integrierten Kalibrierungsfenster von Beam kalibriert wurde. Bei jedem Start des Programms wurde das selbst erstellte Kalibrierungsfenster geöffnet. Allerdings war das selbst entwickelte Kalibrierungsfenster in seiner Größe anpassbar. Dadurch wurden die Punkte im Sichtfeld der Testperson angezeigt und nicht an den Rändern der Leinwand. Eine schematische Abbildung des Kalibrierungsfensters ist in Abbildung 3 zu sehen. In der Realität hatte das Fenster einen weißen Hintergrund, um die Lichtverhältnisse relativ zum Beam Kalibrierungsfenster, welches sehr dunkel ist, zu verbessern. Die Ergebnisse der Kalibrierung wurden anschließend verwendet, um die Rohdaten des Eye Trackers mit diesen zu transformieren, damit die Kalibrierung Einfluss auf die Schätzwerte der Blickposition nimmt. Die daraus resultierenden Ergebnisse wiesen allerdings keine visuell erkennbare Verbesserung auf. Auf Basis der bisherigen Ergebnisse wird daher empfohlen, die integrierte Standard-Kalibrierung des Eye Trackers zu verwenden. Der Code für die selbst entwickelte Kalibrierung ist allerdings weiterhin in auskommentierter Form vorhanden und kann bei Bedarf wieder reaktiviert und weiterentwickelt werden.

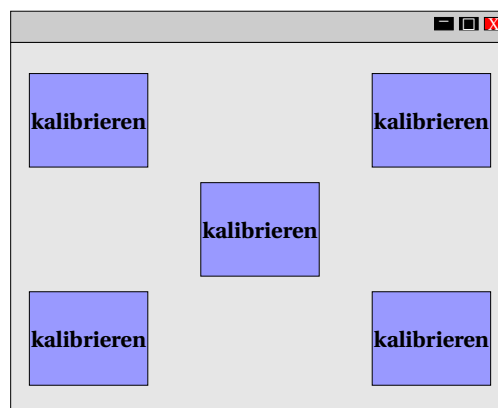


Abbildung 3: Schematische Darstellung des selbst entwickelten Kalibrierungsfensters

Während der Kalibrierung und der gesamten Verwendung des Eye Tracking Systems darf ausschließlich die Testperson im Fahrsimulator sitzen. Eine zweite Person würde das Tracking System beeinflussen, da nicht selektiert werden kann, welche Augen getrackt werden sollen und welche nicht. In Zukunft muss hier eine visuelle Abschirmung, wie eine Art Scheuklappe eingebaut werden, um dieses Problem zu lösen.

Ein weiterer wichtiger Faktor ist die Beleuchtung im Fahrsimulator. Das Kalibrierungsfenster des Beam Eye Trackers hat einen schwarzen Hintergrund. Wenn keine externe Lichtquelle verwendet wird, dann sind die Lichtverhältnisse unzureichend, als dass eine ordentliche Kalibrierung stattfinden könnte. Selbst wenn während der Kalibrierung gute Lichtverhältnisse herrschen, reicht die Helligkeit der Leinwand während des Fahrens nicht aus, um verlässliche Ergebnisse zu liefern. Es konnte eine Verbesserung der Trackingqualität erzielt werden, nachdem im Fahrsimulator eine externe Lichtquelle während der gesamten Kalibrierung und der gesamten Fahrzeit eingesetzt wurde. Neben externen Mitteln, um die Lichtverhältnisse zu verbessern, können auch in der Windows Kamera App Anpassungen vorgenommen werden. Dazu muss in den Einstellungen der Kamera App unter 'Kameraeinstellungen' die Option 'Erweiterte Steuerelemente für Fotos und Videos' aktiviert werden. Dadurch wird ein Helligkeitsregler verfügbar, der die Helligkeit des Kamerabildes systemweit verändert. Je nach aktuellen Lichtverhältnissen kann ein Erhöhen oder Verringern der Bildhelligkeit den Kontrast der Pupillen relativ zur Umgebung verbessern.



Abbildung 4: Positionierung von Kamera und externer Lichtquelle, Quelle: [10]

In Abbildung 4 sind die Kameraposition und die externe Lichtquelle im Fahrsimulator zu erkennen. Licht und Kamera sollten zu einem späteren Zeitpunkt noch besser in das System integriert werden. Zu Testzwecken hat der Aufbau allerdings verlässliche Ergebnisse geliefert.

3.2 Beam Eye Tracker SDK

Eyewear Tech bietet beim Kauf des Beam Eye Tracker das SDK an. Das SDK ermöglicht es, die Ausgabe des Eye Trackers in selbst geschriebenen Anwendungen zu verwenden [11]. Das SDK muss nicht zusätzlich erworben werden, sondern ist im Kaufpreis des Eye Trackers enthalten. Auf der Website des Beam SDK wird in Python geschriebener Beispielcode zur Verfügung gestellt. In diesem Code werden die nötigen Schritte für eine Verbindung mit der API des Eye Trackers durchgeführt. Der Code wurde auch als Grundlage für die Implementierung des Eye Trackers in dem Programm, das im Rahmen dieser Arbeit entwickelt wurde, verwendet [8].

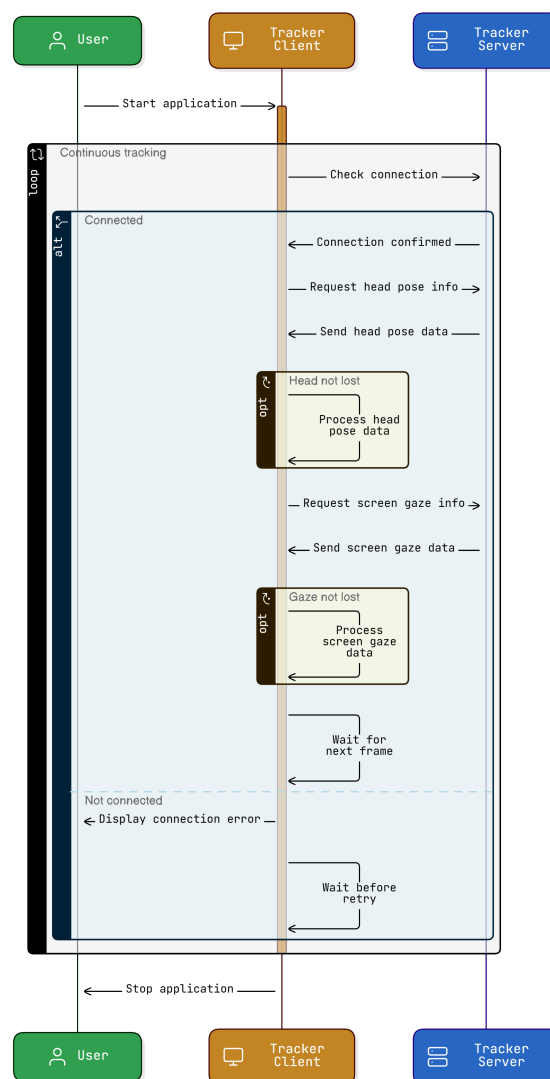


Abbildung 5: Visuelle Darstellung des Codes zur Beam SDK, Quelle: nachempfunden: [8], [12]

Die Funktionsweise der API ist vereinfacht in Abbildung 5 zu erkennen. Der Tracker

Server ist die Bezeichnung für den Eye Tracker selbst. Der Client ist das selbst geschriebene Programm, welches die Trackerdaten über die API entgegennimmt. Die beiden zentralen Informationen, welche vom Server an den Client übergeben werden, sind die Daten zur Kopfposition (head pose data) und die Daten zur Blickposition (screen gaze data). Die Kopfposition gibt Auskunft über die Translation und Rotation des Kopfes der Testperson. Die Blickposition gibt Auskunft darüber, auf welchen Punkt auf dem Bildschirm die Testperson schaut [11]. Diese beiden Systeme laufen unabhängig voneinander und dienen unterschiedlichen Einsatzzwecken. Das Kopftracking kann bspw. dazu dienen, die Kamera in einer Simulationswelt zu bewegen. Das könnte Sinn machen, wenn keine große Leinwand wie im Fahrsimulator vorhanden ist. Durch einen Blick nach links könnte sich auch die Kamera im Fahrsimulator mit nach links bewegen. Für den Einsatzzweck im Fahrsimulator ist diese Funktion durch die große Leinwand allerdings nicht notwendig. Die Daten werden dennoch übermittelt, falls in Zukunft doch ein Anwendungsfall entstehen sollte. Die Gaze Daten stellen im Kontext dieser Arbeit die primär relevante Information dar. Diese geben eine Schätzung ab, auf welchen Punkt die Testperson auf der Leinwand schaut. Sofern eine Testperson vom Eye Tracker erkannt wird, werden in kontinuierlichen Abständen diese Daten vom Eye Tracker an den Client übermittelt. Zusätzlich dazu werden auch noch andere Daten wie bspw. die confidence übermittelt. Dabei handelt es sich um einen Wert, welcher mit jeder geschätzten Blickposition mit übermittelt wird. Dieser gibt an, wie sicher sich der Eye Tracker mit seiner Schätzung der aktuellen Blickposition ist [11].

3.3 Schnittstelle zu CARLA

Im folgenden Abschnitt soll der Datenfluss zwischen CARLA und dem Eye Tracker sowie dem selbstgeschriebenen Programm erläutert werden. Der Eye Tracker und CARLA arbeiten größtenteils getrennt voneinander. Die visuelle Ausgabe des Eye Trackers erfolgt durch einen kleinen schwarzen Punkt, welcher die aktuell geschätzte Blickposition auf der Leinwand anzeigt. Dieser schwarze Punkt befindet sich in einem selbst erstellten transparenten Fenster, welches über den Bildschirm gelegt wird. Der schwarze Punkt ist nicht Teil von CARLA oder dem Eye Tracker sondern wird durch das selbst geschriebene Programm erzeugt. Er basiert auf den x- und y-Koordinaten des Eye Trackers, welche durch eine zusätzliche Korrekturfunktion optimiert werden. Diese Korrektur wird in einem späteren Abschnitt genauer erläutert. Es gibt auch einen sogenannten Textmanager. Dieser ist auch selbst entwickelt und weder Teil von CARLA noch vom Beam Eye Tracker. In diesem werden für den Nutzer unter anderem die grobe Blickrichtung in Textform angezeigt. Zum Beispiel 'schaut nach oben links'. Der Text Manager, wird wie in der Abbildung 6 zu sehen ist, mit Informationen des Eye Trackers und von CARLA gespeist.

Die einzige direkte Schnittstelle zwischen CARLA und den Daten des Eye Trackers ist die Übergabe der Eye Tracker Daten an die Auswertung des Instance Segmentation Sensors. Dieser Sensor wird in einem späteren Abschnitt genauer erläutert. Diese Schnittstelle ist in Abbildung 6 schematisch dargestellt. Der CARLA Server sendet die

Sensordaten des Instance Segmentation Sensors an den CARLA Client. Der CARLA Client übergibt die Sensordaten an die Auswertung. Bei der Auswertung werden die Rohdaten des Eye Trackers, die zuvor durch eine Krümmungs-Korrektur angepasst wurden, dann mit den Sensordaten des Instance Segmentation Sensors verarbeitet. Durch die Kombination aus Eye Tracking Daten und den Daten des Sensors kann eine Aussage darüber getroffen werden, auf welches Objekt die Testperson innerhalb der CARLA Simulationswelt schaut. Diese Ausgabe wird dann im Text Manager in Form von Text auf dem Bildschirm dargestellt. Eine genaue Beschreibung der Funktionsweise der einzelnen Elemente folgt in späteren Abschnitten.

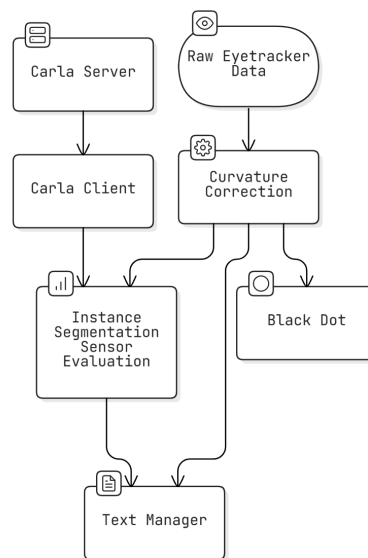


Abbildung 6: Graphische Darstellung der Verarbeitung der Raw-Daten des Eye Trackers, Quelle: [13]

4 CARLA Umgebung

4.1 Aufbau Testszenario in CARLA

Um den Eye Tracker in Zusammenhang mit CARLA testen zu können, muss eine Testumgebung in CARLA angelegt werden. In Abbildung 7 sind die groben Schritte zur Erzeugung einer Testumgebung zu erkennen. Hier ist das Zusammenspiel zwischen dem CARLA Server und dem CARLA Client hervorgehoben. Der Host-PC ist in diesem Fall der Computer, auf welchem der Fahrsimulator ausgeführt wird. Auf diesem ist Windows als Betriebssystem installiert. Im gesamten Zeitraum der Projektarbeit wird der CARLA Server immer lokal auf diesem Computer laufen. Der CARLA Server ist für die Simulation der Welt sowie der Aktoren in ihr verantwortlich. Der Client läuft auf demselben Computer und kann Anfragen an den Server senden. Auf diese Weise kann der Client die Simulation aktiv beeinflussen oder Daten wie bspw. Positionsdaten oder Bilddaten vom Server anfragen. [13]

Der CARLA Server kann heruntergeladen werden und direkt als ausführbares Programm gestartet werden. Anders als der CARLA Client, welcher selbst in Python geschrieben werden muss. Dafür kann dieser auch nach den eigenen Anforderungen individuell aufgesetzt werden. In der Dokumentation von CARLA gibt es einige Beispiele, wie ein Client aufgebaut ist, nach welchen sich der Client für diese Projektarbeit auch orientiert [14]. Zur Serververbindung nutzt er das API des CARLA Servers. Der Client wurde so geschrieben, dass er zuerst versucht, eine Verbindung zum CARLA Server aufzubauen. Anschließend holt er sich wichtige Informationen über die simulierte Welt und mögliche Spawnpunkte für bspw. Fahrzeuge vom Server. Unter dem Begriff Actor werden in diesem Kontext unter anderem Fahrzeuge und Sensoren bezeichnet. Der Client fordert eine Liste aller in der aktuell simulierten Welt vorhandenen Fahrzeuge und Sensoren an und 'zerstört' diese anschließend. Somit wird sichergestellt, dass keine Aktoren aus einer vorherigen Sitzung in die neue Sitzung mitgebracht werden. Nachdem die simulierte Welt wieder einen fest definierten Ausgangszustand angenommen hat, beginnt der Client damit, einen Mercedes Sprinter zu erzeugen. Abbildung 8 stellt die Fortsetzung des Prozesses dar. Nachdem das Fahrzeug vom Server in der Welt erzeugt wurde, fordert der Client die Erzeugung von zwei Sensoren an. Beide Sensoren sollen an das erzeugte Fahrzeug geknüpft sein. Der erste Sensor soll den Rot, Grün, Blau (RGB)-Farbraum erfassen. Der zweite Sensor soll die Instance Segmentation erfassen. Auf diese und die Verarbeitung der daraus resultierenden Daten wird weiter unten genauer eingegangen.

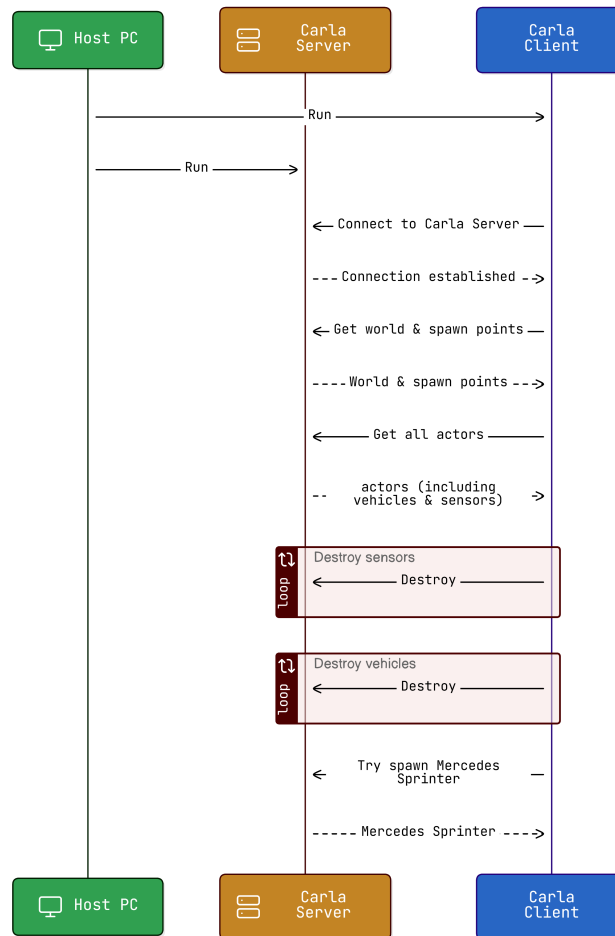


Abbildung 7: Visuelle Darstellung des Codes zur CARLA Testumgebung Teil 1/2, Quelle: [12],[13]

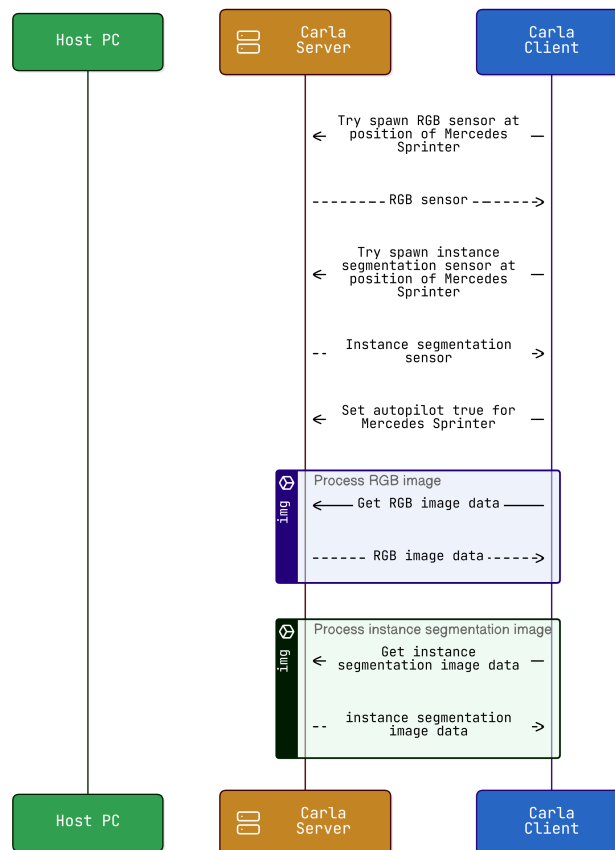


Abbildung 8: Visuelle Darstellung des Codes zur CARLA Testumgebung Teil 2/2, Quelle: [12],[13]

4.2 Sensor Auswertung

4.2.1 Verschiedene Sensortypen

Der RGB-Sensor liefert ein Bild an den Client, welches einer Kameraaufnahme gleicht. Durch die roten, grünen und blauen Farbanteile kann ein Bild erzeugt werden, welches für die Testperson im Fahrsimulator die simulierte Welt in Farbe darstellt. Die simulierte Welt kann durch diesen RGB-Sensor wie durch eine Kameralinse betrachtet werden. Der Server berechnet die Welt in Zahlen und der Sensor ermöglicht es der Testperson im Fahrsimulator, diese auf Zahlen basierende Welt visuell wahrzunehmen. Die Objekte und die Farben der Welt werden so dargestellt, wie sie für den Menschen natürlich sind. Ein Baum hat die Form eines Baums und besitzt grüne Blätter und einen braunen Stamm. In CARLA kann bei Bedarf auch ein Lidar-Sensor platziert werden. Dieser erzeugt ebenfalls ein Bild der simulierten Umgebung. Dieses würde den Baum dann als eine Punktwolke in einer schwarzen Umgebung darstellen. Ein Lidar-Sensor würde die Welt in einer anderen Art und Weise wahrnehmen, wodurch auch ein anderes Bild im Vergleich zu einem RGB-Sensor entstehen würde.

4.2.2 Instance Segmentation Sensor

Der Instance Segmentation Sensor ist in seiner Funktion etwas spezieller. Dieser kategorisiert die Objekte in der Welt in Gruppen ein und weist ihnen Identifikationsnummern (ID)s zu. Er färbt gewisse Objektgruppen nach ihrer jeweiligen Gruppenzugehörigkeit. Bspw. werden Straßen hellgrün und Gebäude dunkelblau gezeichnet. Zusätzlich besitzt jedes Objekt eine eindeutige Identifikationsnummer (ID). So hat jedes Fahrzeug in der simulierten Welt eine eindeutige, ihm zugewiesene ID. Damit sowohl die Gruppenzugehörigkeit, als auch die ID in die Bildinformation der Sensorausgabe übergeben werden können, gibt es zwei Codierungsmechanismen. Bei der Sensorausgabe handelt es sich wie beim RGB-Sensor um eine Farbbilddarstellung, bestehend aus rot-, grün- und blau-Anteilen. Die Zugehörigkeit eines Objekts zu einer bestimmten Gruppe wird durch den Rot-Anteil des Bildes an der Stelle des Objektes bestimmt [13]. Die Tabelle 1 stellt die korrekte Zuweisung zwischen dem Rot-Wert und der jeweiligen Eingruppierung dar. Im Code wurde sich auf diese Zuweisung gestützt. Es muss berücksichtigt werden, dass bei unterschiedlichen Versionen von CARLA eine unterschiedliche Zuweisung erfolgt [13].

Die ID wird in den grün- und blau-Anteil des Bildes codiert [13]. Liegt bspw. ein grün-Anteil von 22 und ein blau-Anteil von 232 vor, dann lautet die ID des Objektes '22-232'. Der Client fordert die Sensorausgabe des Instance Segmentation Sensors in regelmäßigen Abständen vom Server an. Anschließend extrahiert er die RGB-Werte und wertet diese nach den oben beschriebenen Zuordnungen aus. Die Auswertung erfolgt dabei an einem einzigen Pixel. Die Position des Pixels ist definiert durch die aktuelle Ausgabe der geschätzten Blickposition vom Eye Tracker.

Rot-Wert	Gruppe
0	Unlabeled
1	Roads
2	SideWalks
3	Building
4	Wall
5	Fence
6	Pole
7	TrafficLight
8	TrafficSign
9	Vegetation
10	Terrain
11	Sky
12	Pedestrian
13	Rider
14	Car
15	Truck
16	Bus
17	Train
18	Motorcycle
19	Bicycle
20	Static
21	Dynamic
22	Other
23	Water
24	RoadLine
25	Ground
26	Bridge
27	RailTrack
28	GuardRail

Tabelle 1: Zuweisung Rot-Wert und Objektgruppe bei CARLA Version 0.9.15, Quelle: [13]

4.2.3 Optimierung der Performance

Der Rechenaufwand, den der Server leisten muss, um die Sensorausgabe zu generieren, ist nicht zu vernachlässigen. Bei der RGB-Ausgabe sind nicht viele Abstriche möglich, da die Testperson ein Bild von hoher Auflösung für die große Leinwand im Fahr-simulator benötigt. Die Sensorausgabe des Instance Segmentation Sensors kann hin-gegen auch mit einer geringeren Auflösung generiert werden. Dadurch wird Rechen-leistung gespart und da die Sensorausgabe nur im Hintergrund und nicht zur Visua-lisierung benötigt wird, macht es für die Testperson auch keinen bemerkbaren Unter-schied. Die geringere Auflösung hat durch selbst durchgeführte Experimente keinen bemerkbaren Einfluss auf die Qualität der Trackingergebnisse gehabt.

Damit das Zusammenspiel zwischen Eye Tracker und Sensor weiterhin funktioniert, müssen die Werte des Eye Trackers ebenfalls auf die herunter skalierte Sensorausgabe umgerechnet werden. In Abbildung 9 ist diese Skalierung grafisch dar-gestellt. Durch den Faktor zehn werden aus den Originalsensordaten jeweils 10x10 Pi-xel auf ein einzelnes Pixel herunter skaliert. Alle Pixel innerhalb dieses 10x10 Rasters, wie bspw. das Pixel bei Position $x=7$ und $y=4$ werden dann auf ein Pixel bezogen. Da-durch wird die Genauigkeit theoretisch verringert, da der Eye Tracker zwar Tracking Daten bezogen auf die Originalauflösung ausgibt, dann aber auf die herunter skalierte Instance Segmentation Sensorausgabe beziehen muss. In der Realität wurde allerdings keine messbare Verschlechterung der Tracking Ergebnisse festgestellt.

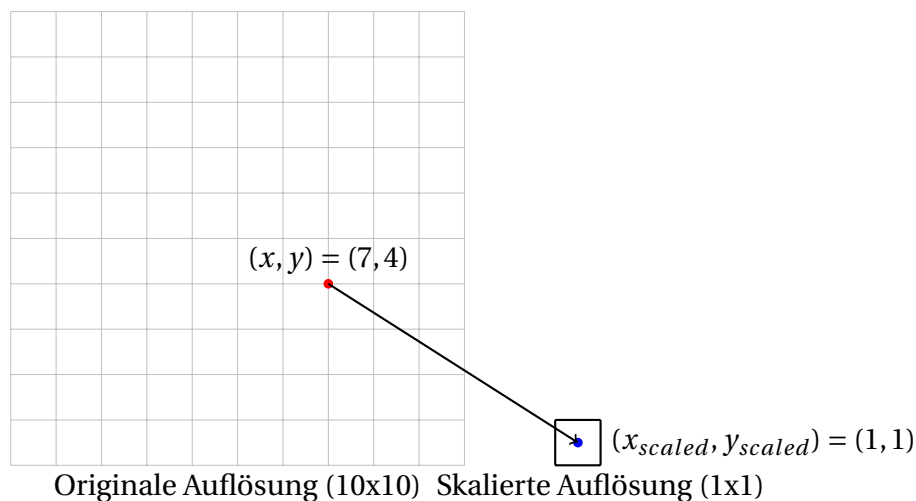


Abbildung 9: Herunterskalierung der Eye Tracking-Daten um Faktor 10

4.3 Darstellung der Daten in CARLA

Die Daten des Eye Trackers werden nicht direkt in CARLA dargestellt, sondern in einem transparenten Windows-Fenster, welches über die CARLA Client Anwendung geschoben wird. Die Inhalte dieses transparenten Fensters können allerdings in Zukunft bei Bedarf in die CARLA Anwendung integriert werden.

In Abbildung 10 ist die graphische Darstellung der Daten des Eye Trackers im transparenten Text Manager zu erkennen. Das erste Anzeigeelement ist der lilagefärbte Kreis. Dieser zeigt die geschätzte Blickposition des Eye Trackers an. Bei den Werten handelt es sich um die Rohdaten, welche noch nicht durch die Korrekturfunktion bearbeitet wurden. Als Zweites wird ein roter Punkt angezeigt. In der Realität ist das der Mauszeiger. Zur besseren Sichtbarkeit wurde dieser durch einen roten Kreis hervorgehoben. Der Mauszeiger wurde dazu verwendet, um die Performance des Eye Trackers messbar zu machen. Die Testperson hat den Mauszeiger bewegt und auf diesen geschaut. Dadurch war der Wert der tatsächlichen Blickposition bekannt und konnte mit dem Wert der geschätzten Blickposition verglichen werden. Während der normalen Verwendung des Eye Trackers spielt dieser aber keine Rolle und wird daher nicht angezeigt. Als drittes Element wird der schwarze Punkt angezeigt. Dieser zeigt analog zum lilafarbene Kreis die geschätzte Blickposition auf der Leinwand. Allerdings werden für die Darstellung des schwarzen Punktes die korrigierten Werte verwendet. Der schwarze Punkt ist dementsprechend der eigentlich geltende Wert. Das vierte Anzeigeelement ist die textbasierte Ausgabe. In dieser werden verschiedene Werte ausgegeben, welche gleich noch detaillierter beschrieben werden. Das kleine türkisfarbene Fenster oben links ist die Ausgabe des Instance Segmentation Sensors. Diese kann bei der Verwendung des Eye Trackers im Hintergrund ausgeführt werden und ist für die Testperson nicht relevant.

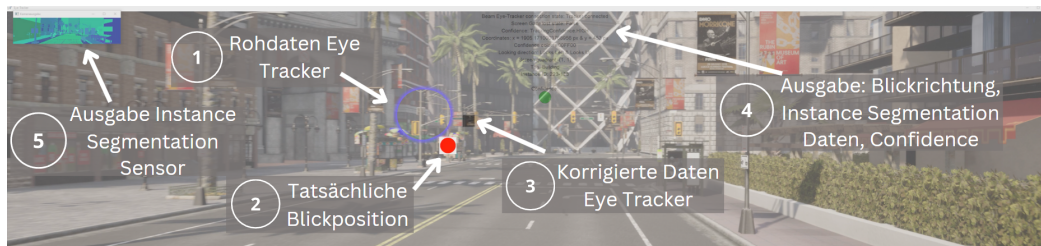


Abbildung 10: Darstellung der Daten in , Quelle: [15], [10]

In Abbildung 11 sind die textbasierten Daten aus Anzeigeelement vier in Abbildung 10 nochmals genauer dargestellt. Es werden Informationen über den Verbindungsstatus des Eye Trackers, die aktuell geschätzten Blickkoordinaten, die grobe Blickrichtung und die Instance Segmentation Auswertung gegeben. Die 'Looking Direction' gibt dabei die grobe Blickrichtung der Testperson an. Der 'Tag' ist die Einordnung des Objektes, auf welches die Testperson schaut, in eine Objektgruppe und die 'Instance ID' die eindeutige Identifikationsnummer für selbiges Objekt. Der grüne Punkt unten gibt die aktuelle Tracking Confidence des Eye Trackers nochmals in Form einer Ampel aus.

Dadurch kann sofort erkannt werden, wenn der Eye Tracker die Testperson nicht mehr korrekt erfasst. Ein Demovideo des Eye Trackers befindet sich im Anhang dieser Arbeit.

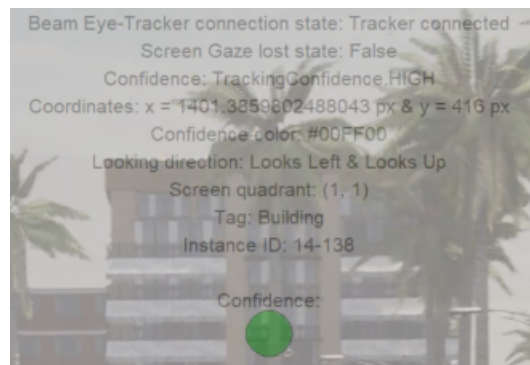


Abbildung 11: Darstellung der textbasierten Daten in , Quelle: [15], [10]

5 Korrektur gekrümmte Leinwand

5.1 Problemursache

5.1.1 Hauptproblem

Die Leinwand des Fahrsimulators ist nicht flach, sondern besitzt eine halbkreisförmige Form. Der Eye Tracker hat keine Einstellmöglichkeit, um eine gekrümmte Leinwand zu berücksichtigen. Dadurch entstehen Abweichungen zwischen der vom Eye Tracker geschätzten Blickposition und der realen Blickposition. Abbildung 12 visualisiert dieses Problem.

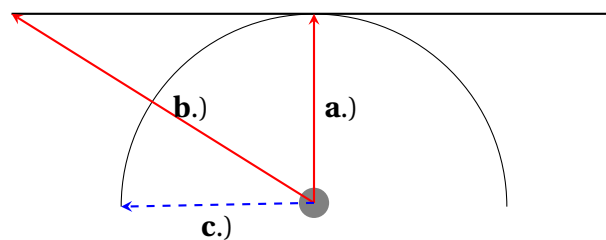


Abbildung 12: Visualisierung der verzerrenden Wirkung einer gekrümmten Leinwand auf die Ergebnisse des Eye Trackers

Grauer Punkt - Sitzposition der Testperson

a.) - Direkte Blickrichtung auf Mitte der Leinwand (0° Winkel)

b.) - Vom Eye Tracker angenommene Blickrichtung zum äußeren linken Rand einer imaginär flachen Leinwand

c.) - Korrekte Blickrichtung relativ zur erfassten Blickrichtung durch Pfeil b.)

In Abbildung 12 sind die gekrümmte Leinwand sowie eine flache Leinwand zu erkennen. Aus Sicht des Eye Trackers existiert nur die flache Leinwand. Ein grauer Punkt kennzeichnet die Position der Testperson im Fahrsimulator. Pfeil a.) zeigt direkt von der Position der Testperson in einem Winkel von 0° auf die Mitte der Leinwand. Die Abbildung verdeutlicht, dass es in diesem Punkt keine Abweichung zwischen der flachen Leinwand und der gekrümmten Leinwand gibt. Aus diesem Grund ist zu erwarten, dass der Eye Tracker in diesem Punkt die genauesten Ergebnisse liefert.

Pfeil b.) zeigt ein zweites Szenario. Die Testperson schaut in diesem Fall nach links. Es ist zu erkennen, dass sie mit ihrem Blick die gekrümmte Leinwand auf der linken Seite etwas links von der Mitte trifft. Wenn der Blick der Testperson allerdings imaginär verlängert wird und dadurch auf die imaginäre, flache Leinwand trifft, wird die Abweichung erkennbar. Es ist zu erwarten, dass der Eye Tracker die Blickposition auf den linken Bildschirmrand schätzt, obwohl die reale Blickposition einen gewissen Abstand

zum Bildschirmrand besitzt. In einem optimierten System würde der Eye Tracker die Blickposition, welche mit Pfeil b.) dargestellt ist, nur dann schätzen, wenn die Testperson auf den Punkt schaut, welcher mit Pfeil c.) dargestellt ist.

5.1.2 Variabilität des Problems

Es besteht kein gleichbleibender Zusammenhang zwischen der Blickposition auf der flachen Leinwand und der gekrümmten Leinwand. In Abbildung 13 sind zwei Beispiele zu erkennen. In Beispiel 1 wird der Blick auf die gekrümmte Leinwand mit dem Pfeil a.) dargestellt. Die Transformation auf die flache Leinwand wird mit Pfeil b.) visualisiert. Bei diesem Beispiel schaut die Testperson in einem Winkel $\alpha = 90^\circ$ nach links. Zwischen dem Blick a.) und dem transformierten Blick b.) stellt sich ein Winkel ein. In Beispiel 2 wird nicht an den Bildschirmrand der gekrümmten Leinwand geschaut, sondern in die Mitte. Der Winkel $\beta = 45^\circ$ stellt sich ein. Auch zwischen dem Blick c.) und dem transformierten Blick d.) stellt sich ein Winkel ein. Aus der Abbildung ist abzulesen, dass der Winkel zwischen b.) und a.) kleiner ist als der Winkel zwischen c.) und d.). Das zeigt, dass die Transformation zwischen der gekrümmten und der flachen Leinwand nicht konstant ist. Das Problem ist an den Rändern stärker ausgeprägt als in der Nähe der Mitte.

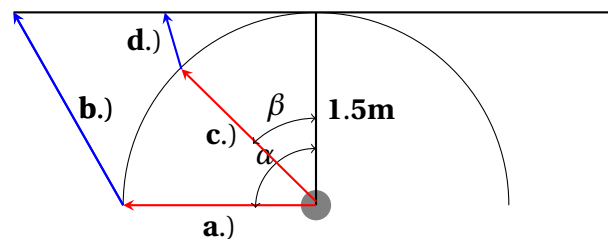


Abbildung 13: Visualisierung der nichtlinearen Transformation

Grauer Punkt - Sitzposition der Testperson

a.) - Blickrichtung um 90° nach links auf den äußeren linken Rand der realen gekrümmten Leinwand

b.) - Transformation von Pfeil a.) auf die imaginäre flache Leinwand

c.) - Blick nach links auf die realen gekrümmten Leinwand

d.) - Transformation von Pfeil c.) auf die imaginäre flache Leinwand

α - Winkel zwischen Blickrichtung von 0° und Pfeil a.)

β - Winkel zwischen Blickrichtung von 0° und Pfeil c.)

5.1.3 Problem der Sitzposition

In den Abbildungen 12 und 13 wurde die Sitzposition der Testperson im Fahrsimulator immer im Mittelpunkt der gekrümmten Leinwand angekommen. In der Realität sitzt die Testperson nicht in der Mitte des Halbkreises, sondern näher in Richtung Leinwand. Ein Mensch hat ein Sichtfeld von ca. 200° [16]. Würde die Testperson in der Mitte des Halbkreises sitzen, dann würde die Leinwand 180° des Sichtfeldes umfassen. Da die Testperson aber näher an der Leinwand sitzt und somit nicht mehr in der Mitte des

Halbkreises, befinden sich die Ränder der Leinwand weiter hinten, wodurch ein größerer Anteil des Sichtfeldes durch die Leinwand abgedeckt wird. Kopfbewegungen im Eye Tracker sind nicht optimal. Da die Ränder der Leinwand nur durch eine kleine Kopfbewegung in das Sichtfeld rutschen, resultiert daraus eine verringerte Genauigkeit des Eye Trackers in der Nähe der Bildschirmränder.

Eine weitere Abweichung von den Annahmen weiter oben ist die um links verschobene Sitzposition der Testperson im Fahrsimulator. Bisher wurde angenommen, dass die Testperson direkt gegenüber der Mitte der Leinwand sitzt. In der Realität ist sie durch das nachgebaute Fahrzeugcockpit leicht nach links versetzt. Dadurch verhält sich die Transformation von flacher auf gekrümmte Leinwand links unterschiedlich als rechts. Abbildung 14 visualisiert diese Einflüsse.

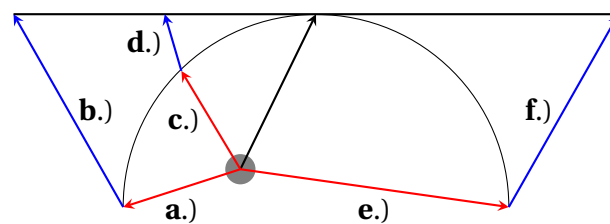


Abbildung 14: Visualisierung der Verschiebung der Sitzposition der Testperson

Grauer Punkt - Sitzposition der Testperson

- a.) - Blickrichtung auf den äußeren linken Rand der realen gekrümmten Leinwand
- b.) - Transformation von Pfeil a.) auf die imaginäre flache Leinwand
- c.) - Blick nach links auf die realen gekrümmten Leinwand
- d.) - Transformation von Pfeil c.) auf die imaginäre flache Leinwand
- e.) - Blickrichtung auf den äußeren rechten Rand der realen gekrümmten Leinwand
- f.) - Transformation von Pfeil e.) auf die imaginäre flache Leinwand

Es ist zu erkennen, dass in diesem Szenario vor allem zwischen der Blickrichtung c.) und der Transformation d.) kaum Fehler besteht. Durch die Sitzposition weiter links wird dort eine geringere Abweichung zwischen gekrümmter und flacher Leinwand bestehen. Experimente am Fahrsimulator haben diese Annahme bestätigt. Besonders in der Mitte der linken Seite hat der Eye Tracker gut geschätzt. Weiter links und sehr weit rechts wurde die Genauigkeit ohne eine Transformation auf die gekrümmte Leinwand wesentlich schlechter.

5.2 Lösungsansatz

Wie im Abschnitt davor bereits beschrieben, muss eine Transformation zwischen den beiden Leinwänden durchgeführt werden. Die Richtung der Transformation wird dabei durch die Datengrundlage bestimmt. Die Daten stammen vom Eye Tracker und dieser geht von einer flachen Leinwand aus. Dementsprechend muss die Transformation von der flachen Leinwand auf die gekrümmte Leinwand stattfinden. Die Sitzposition und der Blinkwinkel bestimmen, wie die Transformation durchzuführen ist.

Bei der Transformation ist nur die Horizontale von Relevanz. Vertikale Pixel müssen nicht transformiert werden, weil die Leinwand zwar horizontal gekrümmt ist aber nicht vertikal. Es besteht kein vertikaler Fehler zwischen der flachen und der gekrümmten Leinwand. Zu Zwecken der Nachvollziehbarkeit werden zu Beginn zwei einzelne Pixel betrachtet, welche von der flachen Leinwand auf die gekrümmte Leinwand projiziert werden sollen. Beide Pixel sollen sich auf der linken Hälfte der flachen Leinwand befinden. Abbildung 15 stellt den resultierenden Fehler bei gegebener Sitzposition und Blickrichtung dar.

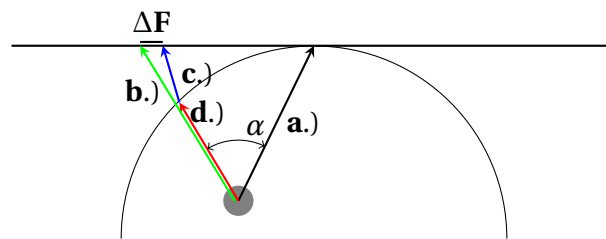


Abbildung 15: Graphische Darstellung des Lösungsansatzes für die Transformation

Grauer Punkt - Sitzposition der Testperson

a.) - Beziehung zwischen Sitzposition der Testperson und Mitte der Leinwand

b.) - Vom Eye Tracker erkannte Blickrichtung auf der imaginären geraden Leinwand, wenn Testperson in Blickrichtung d.) auf gekrümmter Leinwand schaut

c.) - Korrekte Transformation von gekrümmter Leinwand auf gerade Leinwand

d.) - Blick der Testperson auf einen Punkt auf der gekrümmten Leinwand

α - Winkel zwischen Blickrichtung zur Mitte der realen gekrümmten Leinwand und aktueller Blickrichtung

ΔF - Fehler zwischen erkannter Blickrichtung b.) und tatsächlicher Blickrichtung c.) bezogen auf imaginäre gerade Leinwand

In Abbildung 15 ist der Vektor von der Sitzposition zur Mitte der gekrümmten Leinwand durch a.) gegeben. Die Testperson schaut bezogen auf die gekrümmte Leinwand in Richtung Pfeil d.). Der Eye Tracker würde allerdings den Punkt, auf welchen Pfeil b.) zeigt, erkennen. Dieser würde wiederum mit einem anderen Punkt auf dem Halbkreis korrespondieren. Welcher in der Abbildung nicht dargestellt wird, weil es sonst zu unübersichtlich wird. Die Ausgabe des Eye Trackers muss so transformiert werden, dass sie anstelle des in Pfeil b.) dargestellten Punktes den in Pfeil c.) gezeigten Punkt angibt. Es muss also eine Transformation um den Fehler ΔF stattfinden.

Die Berechnung von ΔF ist ein mathematisches Problem. Durch die nach links verschobene Sitzposition und die fehlenden Längenangaben der einzelnen Pfeile ist die Berechnung recht komplex. Ein alternativer Ansatz wäre eine Funktion, welche grob eine Transformation durchführt und vor Ort feinjustiert wird. In Abbildung 16 ist zu erkennen, dass bei einem Blick in die Mitte der linken Hälfte der gekrümmten Leinwand, eine leichte Korrektur nach rechts notwendig ist. Pfeil c.) aus Abbildung 16 verdeutlicht, dass auf der rechten Seite der Halbkugel eine Korrektur nach links notwendig ist und diese stärker ausfallen muss, als auf der linken Seite. Aus den vorherigen Abbildungen ist zudem herauszulesen, dass das Problem umso stärker wird, je näher

sich der Blick den Rändern der Leinwand annähert. Dabei ist es egal, ob linke Hälfte oder rechte Hälfte. Allerdings befindet sich der Punkt, welcher keine Korrektur benötigt, nach wie vor in der Mitte der Leinwand, unabhängig von der Sitzposition der Testperson.

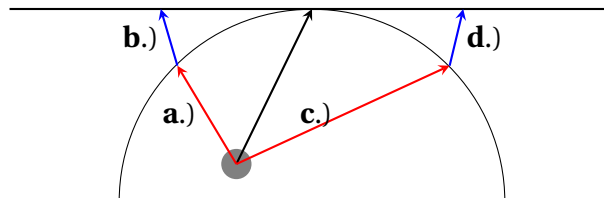


Abbildung 16: Bestimmung der notwendigen Korrektur auf beiden Halbseiten des Halbkreises

Grauer Punkt - Sitzposition der Testperson

- a.) - Blickrichtung auf einen Punkt auf der linken Hälfte der gekrümmten Leinwand
- b.) - Korrekte Transformation von Blickrichtung a.) auf die imaginäre gerade Leinwand
- c.) - Blickrichtung auf einen Punkt auf der rechten Hälfte der gekrümmten Leinwand
- d.) - Korrekte Transformation von Blickrichtung c.) auf die imaginäre gerade Leinwand

Basierend auf diesen Erkenntnissen können mathematische Näherungen aufgestellt werden, mit denen der Fehler reduziert werden soll.

5.2.1 Mathematischer Ansatz e-Funktion

$$k(p) = \begin{cases} e^{c \cdot (w-p)} & \text{für } p \leq w \\ e^{c \cdot (w-(2w-p))} * k_r & \text{für } p > w \end{cases} \quad (1)$$

In einem ersten Ansatz wurde eine Korrekturfunktion verwendet, die auf einer E-Funktion basiert. Die Funktion ist dabei abschnittsweise definiert. Je nachdem ob die Testperson auf die linke oder auf die rechte Bildschirmhälfte schaut, wird eine jeweils andere Korrekturfunktion eingesetzt, um den Korrekturwert zu berechnen. Die abschnittsweise Funktion ist in Gleichung 1 zu erkennen.

Dabei ist:

- k - Korrektur-Wert in Pixeln
- c - Korrektur-Koeffizient (zum Einstellen der Stärke der Korrektur)
- w - Bildschirmbreite in Pixel geteilt durch zwei
- k_r - Zusätzlicher Korrekturfaktor für rechte Bildschirmhälfte
- p - Zu korrigierender Pixel

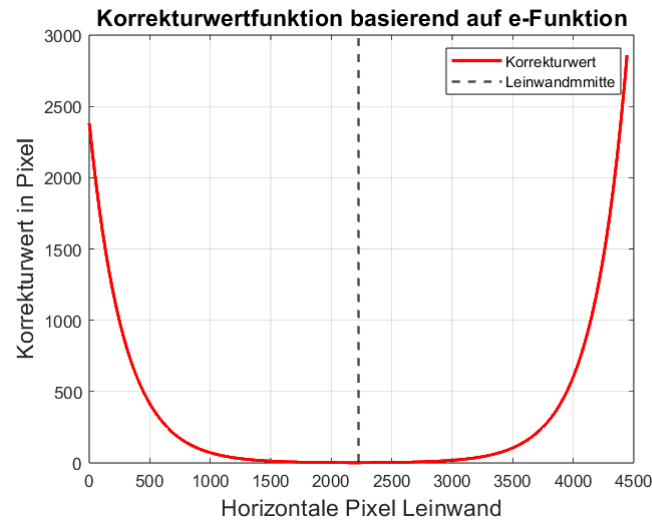


Abbildung 17: Sinnvolle Werte für den Korrekturfaktor linke Bildschirmhälfte

In Abbildung 17 ist zu erkennen, wie sich der Korrekturwert bei einer Leinwand mit 4444 Pixeln Breite bezogen auf die jeweiligen Pixel verhält. Der zusätzliche Korrekturfaktor für die rechte Leinwandhälfte k_r sorgt dafür, dass die Korrektur auf der rechten Hälfte stärker ausgeprägt ist. Das ist notwendig, weil auch der Fehler auf der rechten Bildschirmhälfte stärker auftritt.

Die Feinabstimmung der Parameter wurde experimentell am Fahrsimulator durchgeführt. Die verwendeten Parameter sind im angehängten Matlab-Skript zu erkennen. Mithilfe des Korrekturwertes kann der korrigierte Pixel berechnet werden, wie in Formel 2 zu erkennen ist.

$$p_k = p \pm k \quad (2)$$

Dabei ist:

p_k - Korrigierter Pixel
 p - Zu korrigierender Pixel
 k - Korrektur-Wert in Pixel

Je nachdem, ob sich der aktuelle Pixel auf der linken oder rechten Bildschirmhälfte befindet, wird der Korrekturwert auf den zu korrigierenden Pixel addiert oder von diesem subtrahiert.

Der mathematische Ansatz wurde im Fahrsimulator getestet und es konnten zwar Verbesserungen festgestellt werden, allerdings gab es noch einige Probleme und Optimierungsbedarf. Die Korrektur in der Mitte des Bildschirms ist wesentlich zu schwach. Auch im mittleren Bereich ist sie zu schwach. Zudem sind in Richtung Ränder große Einbußen des Sichtfeldes zu bemängeln. Eine detaillierte Auswertung der Ergebnisse wird hier übersprungen, da schon durch eine erste Beurteilung zu erkennen ist, dass es eine besser geeignete Funktion geben muss.

5.2.2 Limitierung des Sichtbereichs durch Lösungsansatz

Wie bereits beschrieben erzeugt die Funktion aus dem ersten Ansatz ein verringertes Sichtfeld. Das reduziert den effektiven Erfassungsbereich des Eye Trackers im Vergleich zum Zustand ohne Korrektur. Angenommen, die Testperson lenkt ihren Blick immer weiter nach links. Ab einem gewissen Punkt erreicht sie den Punkt $x=0$ (linke Bildschirmhälfte) auf der imaginären flachen Leinwand. Das entspricht in etwa dem letzten Drittel auf der linken Bildschirmhälfte der gekrümmten Leinwand. Ist dieser Punkt erreicht, gibt der Raw-Output des Eye Trackers einen Wert von $x=0$ aus, weil auf einer flachen Leinwand der linke Rand erreicht worden wäre. Bewegt die Testperson ab diesem Punkt den Blick auf der gekrümmten Leinwand noch weiter nach links, dann ändert sich an dem Wert $x=0$ nichts mehr, weil der Eye Tracker denkt der Blick der Testperson wäre bereits am Rand. Parallel zu diesem Phänomen erreicht der berechnete Korrekturwert am Rand sein Maximum. Angenommen, der Korrekturwert würde am Rand einen Wert von 700 Pixel annehmen. Der linke Rand ist auf der flachen Leinwand bei $x=0$ Pixel. Auf der gekrümmten Leinwand ist dieser Wert allerdings weiter rechts. Die linke Bildschirmhälfte umfasst 2222 Pixel. Der Punkt auf der gekrümmten Leinwand liegt in etwa am Ende des ersten Drittels. $\frac{1}{3} \cdot 2222 = 740$ Pixel. Wenn dieser Punkt erreicht ist, dann wird auf diese 740 Pixel der Korrekturfaktor von 700 Pixel addiert. Es ergibt sich eine berechnete Position von $740+700$ Pixel und somit 1140 Pixel. Schaut die Testperson noch weiter nach links, wird das vom Eye Tracker nicht mehr erfasst, weil die detektierte Blickposition bei der flachen Leinwand weiterhin bei $x=0$ ist. Auf der gekrümmten Leinwand bleibt die berechnete Blickposition bei $x=1140$ Pixel hängen. Gleichzeitig bleibt auch der Korrekturwert von 700 Pixel identisch, weswegen eine statische Grenze von 1140 Pixel vom linken Rand entfernt entsteht. Wird der Koeffizient c erhöht, dann wird die Korrekturkurve dadurch steiler. Das wiederum sorgt für eine stärkere Korrekturwirkung. Angenommen, der höhere Korrekturfaktor ist in diesem Beispiel nicht mehr 700 Pixel, sondern 900 Pixel. Die neue statische Grenze liegt jetzt bei $740 \text{ Pixel} + 900 \text{ Pixel} = 1640 \text{ Pixel}$ vom linken Rand entfernt. Das bedeutet, dass ein höherer Korrekturfaktor c die Korrektur stärker macht und gleichzeitig den verwendbaren Bereich des Eye Trackers einschränkt. Auf der rechten Bildschirmhälfte tritt der Effekt noch stärker auf, da dort noch zusätzlich mit dem Korrekturkoeffizienten k_r gerechnet wird.

Hieraus kann das Fazit gezogen werden: Höherer Korrekturkoeffizient c bedeutet eine stärkere Korrektur und möglicherweise bessere Ergebnisse. Gleichzeitig wird aber der Sichtbereich, in welchem der Eye Tracker eingesetzt werden kann, verschmälert. Dieses Problem wird auch bei anderen Korrekturfunktionen bestehen.

5.2.3 Quadratische-Funktion

Anstatt einer e-Funktion wird eine Quadratische Funktion eingesetzt. Das k_r ist wie bei der e-Funktion nur auf der rechten Bildschirmhälfte anzuwenden.

$$k(p) = \begin{cases} (c * (w - p))^2 & \text{für } p \leq w \\ (c * (w - (2 * w - x)))^2 * k_r & \text{für } p > w \end{cases} \quad (3)$$

Die Korrektur mit der neuen Funktion wurde im Labor experimentell getestet. Dabei wurde auch Fine-Tuning durchgeführt, um die Funktion so gut es geht an die realen Gegebenheiten anzupassen. Die Kombination der Parameter $c=0.012$ und $k_r = 1.5$ lieferte die geringste Abweichung im Versuch.

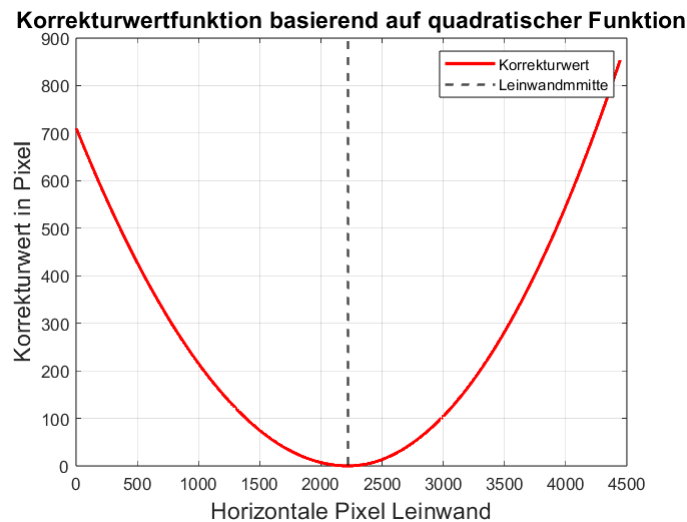


Abbildung 18: Visualisierung quadratische Korrekturfunktion, Quelle: [17]

Abbildung 18 zeigt, dass im Vergleich zur e-Funktion eine stärkere Korrektur in der Nähe der Mitte der Leinwand stattfindet. An den Rändern ist der Korrekturwert nicht so hoch wie bei der e-Funktion, wodurch der nutzbare Sichtbereich weniger stark eingeschränkt wird.

Im Fahrsimulator konnte eine leichte Verbesserung nachgewiesen werden. Hierzu wurde eine Auswertung der Ergebnisse durchgeführt. Kalibriert bedeutet in diesem Fall, dass eine Korrektur der Krümmung vorgenommen wurde.

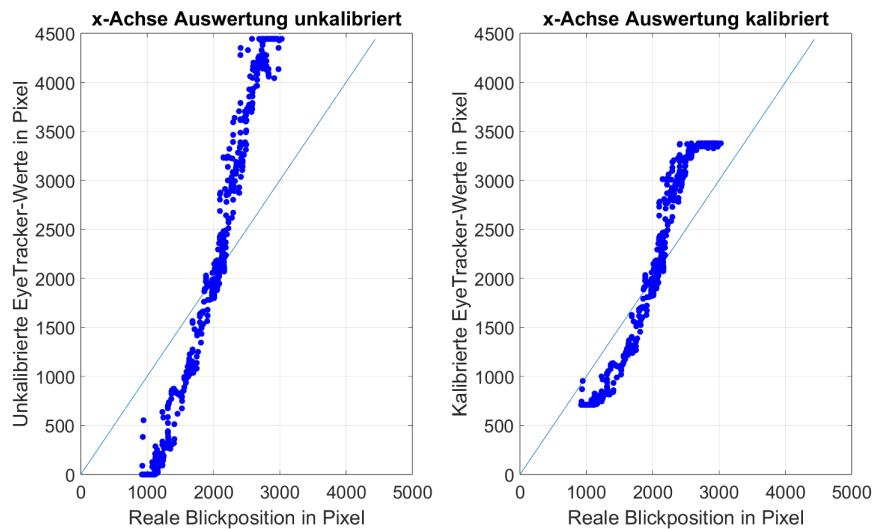


Abbildung 19: Gegenüberstellung reale Blickposition und Eye Tracker-Werte: unkaliibriert (links), kalibriert (rechts), Quelle: [17]

In Abbildung 19 sind die Ergebnisse zu erkennen. Auf der x-Achse ist die reale Blickposition in Pixel aufgetragen. Dabei entspricht 0 Pixel dem linken Bildschirmrand und 4444 Pixel dem rechten Bildschirmrand. Die reale Blickposition wurde mithilfe des Mauszeigers ermittelt. Die Testperson hat den Mauszeiger über den Bildschirm bewegt und ihn mit den Augen verfolgt. Auf der y-Achse ist die vom Eye Tracker geschätzte Blickposition aufgetragen. Diese ist zur Vergleichbarkeit ebenfalls in Pixel angegeben. Für beide Graphen in Abbildung 19 werden auf der x-Achse dieselben Werte verwendet. Auf der y-Achse werden beim linken Graphen die Rohdaten des Eye Trackers aufgetragen. Beim rechten Graphen werden die von der quadratischen Funktion kalibrierten Werte angezeigt. Im Idealfall sollten die blauen Punkte deckungsgleich mit der hellblauen monoton steigenden Gerade sein. Um zu prüfen, dass die Kalibrierung mit der quadratischen Funktion gut funktioniert, sollten die blauen Punkte im rechten Graphen zum einen so nahe wie möglich an der blauen Gerade anliegen und zum anderen bessere Ergebnisse als im linken Graphen erzielen.

Es ist zu erkennen, dass der Sichtbereich auch bei der Verwendung einer quadratischen Korrekturfunktion eingeschränkt wird. Diese Einschränkung konnte allerdings reduziert werden. Bei der e-Funktion war der maximale Korrekturwert auf der linken Bildschirmhälfte $k=2400$. Bei der abgestimmten, quadratischen Korrekturfunktion liegt das Maximum bei $k=700$. Es stehen somit auf der linken Bildschirmhälfte 1800 Pixel mehr für das Tracking zur Verfügung. Die Auswertung stimmt mit den theoretischen Überlegungen soweit überein, dass bei ca. 2222 Pixel ein Fehler von null vorhanden ist. Das liegt daran, dass die gekrümmte Leinwand und eine äquivalente, imaginäre, gerade Leinwand in der Bildschirmmitte bei 2222 Pixel identisch sind. Um die Verbesserungen deutlich zu erkennen, müssen bestimmte Bereiche genauer betrachtet werden:

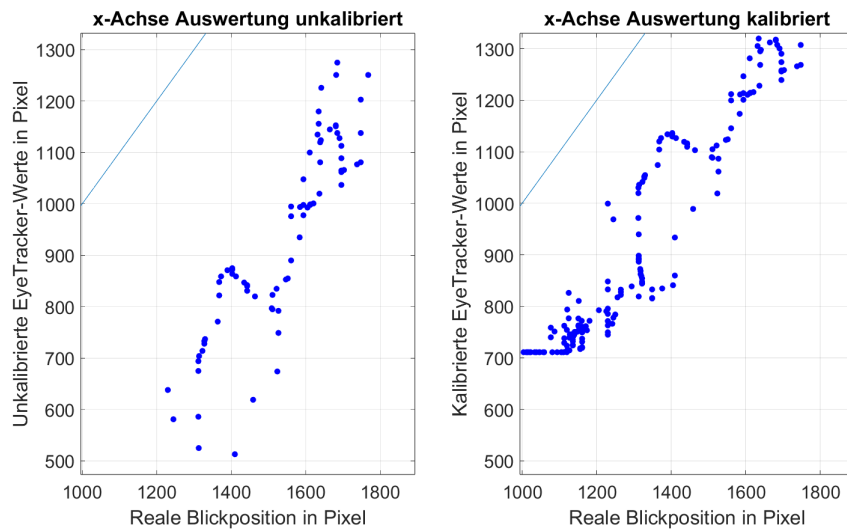


Abbildung 20: Gegenüberstellung reale Blickposition und Eye Tracker-Werte im Detail: unkalibriert (links), kalibriert (rechts), Quelle: [17]

In Abbildung 20 ist ein Detail-Ausschnitt der Ergebnisse der linken Leinwand-Hälfte zu erkennen. Es ist deutlich zu sehen, dass die Werte nach der Kalibrierung näher an der idealen Geraden liegen. Je näher sich die Blickposition dem linken Bildschirmrand nähert, umso geringer wird der absolute Fehler. Es ist festzustellen, dass in Richtung Bildschirmmitte weiterhin zu wenig korrigiert wird. Dort ist ein geringer Fehler zwischen gekrümmter und gerader Leinwand zu erwarten. Dennoch ist der dort fast so hoch wie in Randnähe. Würde der Parameter c erhöht werden, dann würde der Fehler reduziert werden. Gleichzeitig würde der nutzbare Erfassungsbereich des Eye Trackers reduziert werden. Es bestehen dieselben Probleme wie bei der e -Funktion, nur weniger stark ausgeprägt.

Eine Funktion 4. Grades scheint auf den ersten Blick sinnvoll. Diese steigt zwar stärker an als eine quadratische Funktion, allerdings besitzt sie in ihrem Ursprung ein Plateau. Dadurch wäre die Korrektur im Bereich der Bildschirmmitte nicht ausreichend.

5.2.4 V-Funktion

Nach den Experimenten im Fahrsimulator wurde aus den daraus gewonnenen Erfahrungen eine Funktion gesucht, welche in der Bildschirmmitte einen Korrekturwert von null ausgibt. Die Funktion sollte einen Korrekturfaktor ausgeben, welcher sich relativ zum Abstand zu der Bildschirmmitte erhöht. Dabei sollte dieser Anstieg bereits unmittelbar um die Bildschirmmitte hoch sein, damit der Fehler dort bereits ausreichend verringert wird. An den Rändern soll sie allerdings nicht zu sehr ansteigen, da sonst der nutzbare Sichtbereich reduziert werden würde. Dieser sollte im Idealfall größer sein, als bei der quadratischen Funktion. Als Kurvendiagramm sollte die Funktion wie ein V aussehen, dessen Tiefpunkt auf der x-Achse bei der Bildschirmmitte, also bei 2222 Pixeln liegt. Mithilfe von Geogebra wurde eine solche Funktion zusammengebaut [18].

Damit die Steigung stufenlos angepasst werden kann, wurde eine Variable p dafür verwendet. Damit keine negativen Werte herauskommen, muss mithilfe des Betrags sichergestellt werden, dass innerhalb der Basis keine negativen Werte vorkommen. Damit die V-Funktion ihren Tiefpunkt in der Bildschirmmitte hat, muss sie um die Bildschirmmitte nach rechts verschoben werden:

$$k = |x - w|^p \quad (4)$$

Zusätzlich wird der Faktor c eingeführt, welcher ähnlich wie p die Steigung relativ zum Abstand zur Bildschirmmitte beeinflusst. Allerdings nicht ganz so aggressiv wie p . Ein Parameter d wurde eingeführt, um ein Grund-Offset zu erzeugen, welches bei der Feinabstimmung evtl. nützlich sein könnte.

$$k = (c * |(x - w)| + d)^p \quad (5)$$

Damit sich die Ränder der Funktion gegen einen bestimmten Wert annähern, muss eine Dämpfung in die Funktion eingebaut werden. Diese wird durch eine e-Funktion realisiert. Dadurch ist eine schnelle Annäherung an den Grenzwert sichergestellt. Die vollständige Korrekturfaktor-Funktion ist in Gleichung 6 aufgeführt. Es ist zu beachten, dass es beim Ansatz mit der V-Funktion unterschiedliche Parameter für die linke und rechte Seite der Bildschirmhälfte gibt. Diese sind mit einem herab gestellten l bzw. r gekennzeichnet. Die zugewiesenen numerischen Werte sind im angehängten Matlab-Skript aufgeführt.

$$k(p) = \begin{cases} (c_l * |(w - x)| + d_l)^{p_l} * (1 - e^{-g_l * (|w - x|)}) & \text{für } p \leq w \\ (c_r * |(w - (2 * w - x))| + d_r)^{p_r} * (1 - e^{-g_r * (|w - (2 * w - x)|)}) & \text{für } p > w \end{cases} \quad (6)$$

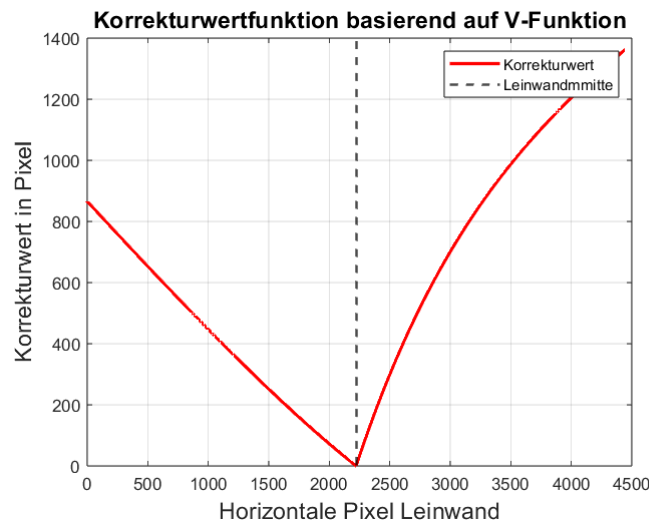


Abbildung 21: Visualisierung V-Korrekturfunktion, Quelle: [17]

Die graphische Darstellung der Korrekturfunktion ist in Abbildung 21 zu erkennen. Die optimalen Parameter wurden im Fahrsimulator experimentell bestimmt und werden

für die Darstellung des Graphen verwendet. Wie in der Abbildung zu erkennen ist, unterscheiden sich die Parameter zwischen linker und rechter Leinwandhälfte. Auf der rechten Hälfte muss stärker korrigiert werden. Hier ist die Veränderung der Steigung ab einem gewissen Punkt gut zu erkennen.

Die Auswertung der Ergebnisse erfolgt im nächsten Abschnitt.

6 Auswertung

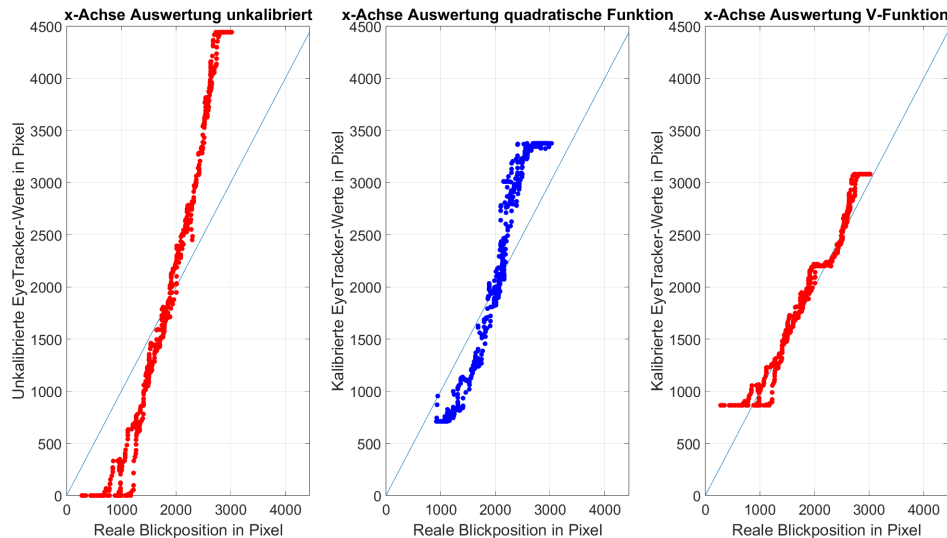


Abbildung 22: Gegenüberstellung reale Blickposition und Eye Tracker-Werte: unkalibriert (links), kalibriert quadratische Funktion (mitte), kalibriert V-Funktion (rechts), Quelle: [17]

In Abbildung 22 sind die Ergebnisse zu erkennen, welche mit der neuen V-förmigen Korrekturfunktion erzielt werden konnten. Es ist eine signifikante Verbesserung gegenüber der quadratischen Korrekturfunktion zu erkennen. Ein eingeschränkter Sichtbereich ist weiter vorhanden. Da ab einem gewissen Punkt allerdings sowieso keine verwertbaren Werte mehr geliefert werden, stellt dies keine relevante Einschränkung für die Anwendung dar. Der Eye Tracker korrigiert in der Nähe der Bildschirmmitte jetzt stärker, was zu einer verbesserten Schätzgenauigkeit führt. Auch in Randnähe wird stärker korrigiert, was zu einer Reduktion des Fehlers führt. Durch die unterschiedlichen Parameter für linke und rechte Bildschirmhälfte entsteht im Übergang von linker auf rechte Bildschirmhälfte und andersherum ein diskontinuierlicher Übergang. Eine weitere Optimierung ist im Rahmen dieser Arbeit nicht vorgesehen.

Zur numerischen Auswertung der Ergebnisse wurde in Matlab ein Skript geschrieben, welches den Fehler zwischen der tatsächlichen Blickposition und der vom Eye Tracker geschätzten Blickposition bzw. der durch den Algorithmus korrigierten Blickposition berechnet. Dieses Matlab-Skript und die aufgezeichneten Messwerte sind dem Anhang beigelegt. Der Fehler wird aufsummiert und anschließend durch die Anzahl der er-

fassten Datenpunkte geteilt. Dadurch ergibt sich ein Durchschnittsfehlerwert. Die Aufzeichnung wurde vor Ort im Fahrsimulator durchgeführt und dauerte zwei Minuten. Dabei wurde die gesamte Leinwand bis auf die äußersten Ränder mit dem Blick abgeflogen. Vor der Auswertung der Datensätze müssen diese vorher bereinigt werden. Wie oben bereits beschrieben, wurde der Mauszeiger als Instrument verwendet, um die tatsächliche Blickposition zu erfassen. Beim Starten und Beenden des Mastrackings wurde allerdings nicht darauf geachtet, auf den Mauszeiger zu schauen. Aus diesem Grund befinden sich in den ersten und letzten paar Sekunden der Datenaufzeichnung große Abweichungen. In Abbildung 23 ist dieser Messfehler als extremer Ausreißer zu erkennen. Diese sind für die Bewertung nicht repräsentativ und werden als Messfehler klassifiziert und aus den Datensätzen bereinigt. Aus diesem Grund wurden die ersten paar hundert und die letzten paar hundert Datenpunkte aus der Aufzeichnung gelöscht, bevor die Auswertung des Fehlers durchgeführt wurde.

Beim nicht kalibrierten Eye Tracker, also den Rohdaten des Beam Eye Trackers liegt auf der x-Achse eine durchschnittliche Abweichung von 697,87 Pixeln vor. Beim Anwenden der quadratischen Korrekturfunktion liegt eine durchschnittliche Abweichung von 368,42 Pixel vor. Die V-Funktion ist in der Lage, den durchschnittlichen Pixelfehler auf der x-Achse auf 107,59 Pixel zu beschränken. Die Ergebnisse zeigen, dass mit der V-Funktion bessere Ergebnisse erzielt werden können als mit der quadratischen Funktion. Zudem wird gezeigt, dass durch den Einsatz zusätzlicher Korrekturfunktion bessere Ergebnisse als mit den Rohdaten des Eye Trackers erzeugt werden können. Dabei ist zu beachten, dass die Erfassung der Daten nicht ideal ist. Die Ergebnisse erlauben dennoch eine erste Einschätzung der Wirksamkeit der Korrekturfunktion. Für eine genauere Bewertung müssten viele Testläufe mit vorher genau definierten Abläufen durchgeführt werden. Bspw. sollte jede Datenaufzeichnung gleich lang gehen und die Testperson sollte in jeder Aufzeichnung die gleichen Bildschirmabschnitte für die gleiche Zeit betrachten. Das würde allerdings den Rahmen dieser Projektarbeit übersteigen.

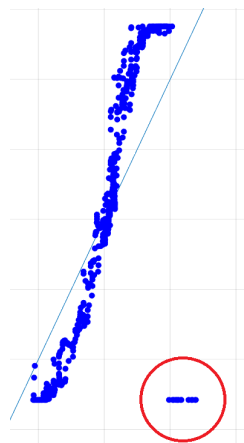


Abbildung 23: Messfehler durch nicht Verfolgen des Mauszeigers mit den Augen zu Beginn und Ende der Messung (unten rechts), Quelle: [17]

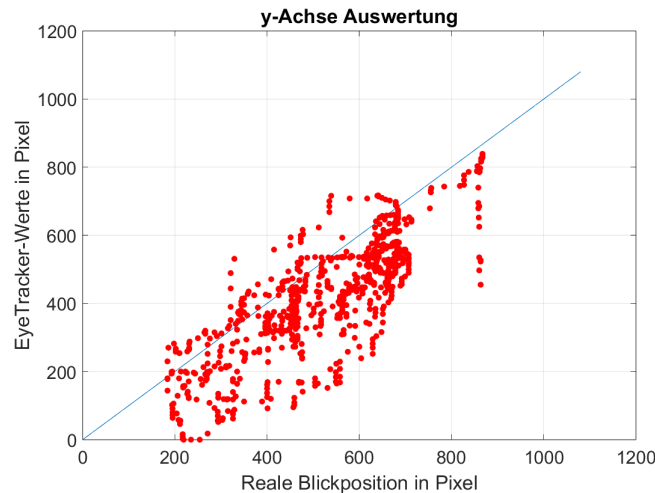


Abbildung 24: Gegenüberstellung reale Blickposition und Eye Tracker-Werte y-Achse, Quelle: [17]

In Abbildung 24 ist die Auswertung der y-Achse nach der Kalibrierung dargestellt. Die Visualisierung lässt zunächst den Eindruck entstehen, als wären die Tracking-Ergebnisse auf der y-Achse schlechter als auf der x-Achse. Das liegt allerdings zum Teil an der anderen Skalierung der y-Achse im Diagramm. Bei den horizontalen Pixelwerten in den Abbildungen weiter oben war die y-Achse von null bis 4444 skaliert. Die y-Achse für die vertikalen Pixelwerte ist allerdings nur von null bis 1080 skaliert. Eine Abweichung von bspw. 300 Pixeln wirkt im Graphen für die vertikalen Pixel größer als im Graphen für die horizontalen Pixel. Ein anderer Grund ist, dass keinerlei Optimierung der vertikalen Pixel durch unseren Code vorgenommen wird. Für die Optimierung der horizontalen Pixel durch die Krümmung der Leinwand wurden große Aufwände betrieben. Durch eine Optimierung der vertikalen Pixel könnten evtl. noch weitere Verbesserungen erzielt werden. Falls es zu vertikalen Abweichungen kommt, dann ist das häufig auf eine Verschiebung der Sitzposition zurückzuführen. Wenn die Testperson bspw. mit der Zeit in den Sitz sinkt. Diese Abweichung ist dann linear. Eine numerische Beurteilung ergibt eine mittlere Abweichung von 120 Pixeln. Relativ zur Bildschirmhöhe ist das ein größerer Fehler als auf der x-Achse. Die Abweichung resultiert jedoch aus absoluten Positionsunterschieden und nicht aus relativen Skalierungsfehlern. Aus diesem Grund ist der Fehler im Vergleich mit der Abweichung von durchschnittlich 107,59 Pixel bei der Verwendung der V-Funktion auf der x-Achse vergleichbar.

7 Fazit

7.1 Auswertung und Zusammenfassung

Die im Rahmen der Projektarbeit definierten Zielsetzungen wurden erfüllt. Die Ergebnisse zeigen, dass es mit handelsüblicher Hardware (USB-Webcam) und sehr kostengünstiger Software möglich ist, bereits verwertbare Ergebnisse zu generieren. Ursprünglich war angedacht, eine Open-Source-Lösung als Fundament für den Eye Tracker zu verwenden. Nach ausgiebiger Marktrecherche konnte ein passendes Softwarefundament ausfindig gemacht werden. Der Beam Eye Tracker ist zwar nicht Open Source, aber die Daten, die er durch seine API bereitstellen kann, bieten genug Spielraum, um individuelle Lösungen damit bauen zu können. Eine Verschmelzung zwischen CARLA und dem Eye Tracker ließ sich ohne signifikante technische Herausforderungen realisieren. Die Daten aus dem Instance Segmentation Sensor können zusammen mit den Werten des Eye Trackers ausgewertet werden. Somit ist es möglich, nicht nur die geschätzten Koordinaten des Eye Trackers auf der Leinwand auszugeben, sondern auch das betrachtete Objekt innerhalb der CARLA Simulationsumgebung zu bestimmen. Während der Implementierung des Eye Tracking Systems sind an manchen Stellen auch Probleme aufgetreten. Die Belichtung war vor allem während der Kalibrierung ein Problem. Durch verschiedene Ansätze und Experimente im Fahrsimulator konnten hier mit externer Belichtung gute Ergebnisse erzielt werden. Eine weitere Herausforderung stellten die durch die gekrümmte Leinwand verfälschten Tracking Daten dar. Durch umfassende Analyse des Problems, einige mathematische Ansätze und Experimente im Fahrsimulator, konnte die daraus resultierende Abweichung signifikant reduziert werden. Die in Abbildung 22 dargestellten Ergebnisse zeigen, dass die korrigierten Daten eine deutlich verbesserte Genauigkeit aufweisen. Die Tracking Daten sind allerdings nicht perfekt und lassen sich durch äußere Einflüsse wie schlechte Belichtung, Bedienungsfehler bei der Kalibrierung, Veränderung der Sitzposition nach der Kalibrierung oder zwei Personen gleichzeitig im Fahrsimulator negativ beeinflussen. Der Eye Tracker könnte allerdings zum aktuellen Entwicklungsstand bereits funktional im Fahrsimulator eingesetzt werden. Anhand der gemessenen Performance, deuten die Ergebnisse darauf hin, dass bereits mit dem aktuellen Entwicklungsstand verwertbare Blickdaten generiert werden können. Im Ausblick sollen weitere Schritte und mögliche Ideen präsentiert werden, mit welchen über diesen Low Budget Ansatz hinaus noch bessere Ergebnisse erzeugt werden könnten.

7.2 Ausblick

Aufbauend auf den Ergebnissen dieser Arbeit kann eine weitere Verbesserung des Testaufbaus erfolgen. Dazu zählen eine Ausbesserung der Lichtverhältnisse im Fahrsimulator, eine fest verbaute Kamera und gegebenenfalls eine Blende, sodass nur der Fahrer und nicht zusätzlich der Beifahrer von der Kamera erkannt wird. Ebenso kann die Eye Tracker Genauigkeit verbessert werden, indem die Korrektur für den gekrümmten Bildschirm durch einen potentiell noch besseren Optimierungsalgorithmus durchgeführt wird. Eine weitere Optimierungsmöglichkeit stellt die Anpassung der vertikalen Kalibrierung dar. Bspw. ließe sich dadurch ein statisches Offset korrigieren, wenn die Person während einer Fahrt in vertikale Richtung die Sitzposition verändert. Eine Integration von künstlicher Intelligenz zur Bestimmung der genauen Blickposition könnte ebenfalls die Genauigkeit verbessern. Abschließend bieten sich weiterführende Analysen an, wie eine zeitabhängige Auswertung von Blickpositionen in einem Zeitstrahl. Der Code des Eye Trackers ist dem Anhang beigelegt und kann somit weiterentwickelt werden.

A Code des Eye Trackers

Der Python Code des Eye Trackers ist dem Projekt als digitaler Anhang beigelegt. Es wurde darauf verzichtet diesen hier einzufügen. Er befindet sich unter Anhänge -> Anhang A Code des Eye Trackers.

B Matlab Skripte & Messwerte

Der Code der Matlab Skripte zur Auswertung der Ergebnisse und zur Darstellung der Korrekturfunktionen sowie die Messergebnisse sind dem Projekt als digitaler Anhang beigelegt. Es wurde darauf verzichtet diesen hier einzufügen. Die Dateien befindet sich unter Anhänge -> Anhang B Matlab Skripte & Messwerte.

C Demo Video Eye Tracker

Ein Demo Video wurde angehängt. In diesem ist eine Bildschirmaufnahme mit aktivem Eye Tracker zu sehen. Der rote Punkt stellt den Mauszeiger dar. Die Testperson hat den roten Punkt mit den Augen verfolgt. Aus diesem Grund kann der rote Punkt mit der tatsächlichen Blickposition der Testperson gleichgesetzt werden. Der lila Kreis ist die Schätzung des Eye Trackers ohne Korrektur. Der schwarze Punkt ist die Schätzung des Eye Trackers mit Korrektur, um die Krümmung der Leinwand auszugleichen. Das Demovideo befindet sich unter Anhänge -> Anhang C Demo Video Eye Tracker.

Abbildungsverzeichnis

1	Ausgabe des Face-Gaze-Trackers, Quelle: [6]	4
2	Ausgabe des Eyegaze-Trackers, Quelle: [7]	5
3	Schematische Darstellung des selbst entwickelten Kalibrierungsfensters	8
4	Positionierung von Kamera und externer Lichtquelle, Quelle: [10]	9
5	Visuelle Darstellung des Codes zur Beam SDK, Quelle: nachempfunden: [8], [12]	10
6	Graphische Darstellung der Verarbeitung der Raw-Daten des Eye Trackers, Quelle: [13]	12
7	Visuelle Darstellung des Codes zur CARLA Testumgebung Teil 1/2, Quel- le: [12],[13]	14
8	Visuelle Darstellung des Codes zur CARLA Testumgebung Teil 2/2, Quel- le: [12],[13]	15
9	Herunterskalierung der Eye Tracking-Daten um Faktor 10	18
10	Darstellung der Daten in , Quelle: [15], [10]	19
11	Darstellung der textbasierten Daten in , Quelle: [15], [10]	20
12	Visualisierung der verzerrenden Wirkung einer gekrümmten Leinwand auf die Ergebnisse des Eye Trackers	21
13	Visualisierung der nichtlinearen Transformation	22
14	Visualisierung der Verschiebung der Sitzposition der Testperson	23
15	Graphische Darstellung des Lösungsansatzes für die Transformation	24
16	Bestimmung der notwendigen Korrektur auf beiden Halbseiten des Halb- kreises	25
17	Sinnvolle Werte für den Korrekturfaktor linke Bildschirmhälfte	26
18	Visualisierung quadratische Korrekturfunktion, Quelle: [17]	28
19	Gegenüberstellung reale Blickposition und Eye Tracker-Werte: unkali- briert (links), kalibriert (rechts), Quelle: [17]	29
20	Gegenüberstellung reale Blickposition und Eye Tracker-Werte im Detail: unkalibriert (links), kalibriert (rechts), Quelle: [17]	30
21	Visualisierung V-Korrekturfunktion, Quelle: [17]	31
22	Gegenüberstellung reale Blickposition und Eye Tracker-Werte: unka- libriert (links), kalibriert quadratische Funktion (mitte), kalibriert V- Funktion (rechts), Quelle: [17]	33
23	Messfehler durch nicht Verfolgen des Mauszeigers mit den Augen zu Be- ginn und Ende der Messung (unten rechts), Quelle: [17]	34

24	Gegenüberstellung reale Blickposition und Eye Tracker-Werte y-Achse, Quelle: [17]	35
----	--	----

Abkürzungsverzeichnis

IEEM Institut für energieeffiziente Mobilität

API Application Programming Interface

IR Infrarot

SDK Software Development Kit

USB Universal Serial Bus

HKA Hochschule Karlsruhe

RGB Rot, Grün, Blau

ID Identifikationsnummer

Glossar

Actor Oberbegriff für Sensoren, Fahrzeuge etc. im CARLA-Ökosystem. 13

Bildwiederholrate Wichtiges Kriterium bei Kameras oder Bildschirmen. Gibt an wie viele Bilder pro Sekunde aufgenommen bzw. dargestellt werden. 4

CARLA Open-Source-Simulationsumgebung für autonomes Fahren und Verkehrsszenarien. 1

e-Funktion Mathematische Exponentialfunktion, verwendet zur Modellierung von Anstiegsverhalten. 25

Human in the Loop Der Mensch ist aktiv am Entscheidungsprozess beteiligt, indem er mit der simulierten oder automatisierten Umgebung interagiert und diese beeinflusst. 1

Instance Segmentation Verfahren zur Erkennung und Klassifizierung einzelner Objekte in digitalen Bildern. 2

Lidar Alternativer Sensortyp basierend auf Lasertechnologie. 16

Open Source Softwarelizenzmodell – hilfreich zur Abgrenzung gegenüber kommerziellen Systemen. 4

Pixel Kleinste Einheit eines digitalen Bildes, oft in Bezug auf Bildschirmkoordinaten. 24

Quadratische Funktion Mathematische Exponentialfunktion, verwendet zur Modellierung von Anstiegsverhalten. 28

Raw-Output Unverarbeitete Ausgabedaten des Eye Trackers. 27

Server-Client Struktur Architekturmodell, bei dem ein Server zentrale Ressourcen verwaltet und Clients darauf zugreifen. 1

V-Funktion Mathematische Exponentialfunktion, verwendet zur Modellierung von Anstiegsverhalten. 31

Symbolverzeichnis

k	Korrekturwert in Pixeln
p	Zu korrigierender Pixelwert auf der x-Achse
p_k	Korrigierter Pixelwert auf der x-Achse
w	Halbe Bildschirmbreite in Pixeln
c	Korrektur-Koeffizient zur Einstellung der Korrekturstärke
k_r	Zusätzlicher Korrekturfaktor für die rechte Bildschirmhälfte
c_l	Korrektur-Koeffizient für die linke Bildschirmhälfte (V-Funktion)
c_r	Korrektur-Koeffizient für die rechte Bildschirmhälfte (V-Funktion)
d_l	Offsetwert für die linke Bildschirmhälfte (V-Funktion)
d_r	Offsetwert für die rechte Bildschirmhälfte (V-Funktion)
p_l	Potenzwert zur Formanpassung der Funktion links (V-Funktion)
p_r	Potenzwert zur Formanpassung der Funktion rechts (V-Funktion)
g_l	Dämpfungsfaktor für die linke Bildschirmhälfte (V-Funktion)
g_r	Dämpfungsfaktor für die rechte Bildschirmhälfte (V-Funktion)
x	Aktuelle horizontale Blickposition der Testperson
α	Winkel zwischen Blickrichtung zur Leinwandmitte und tatsächlicher Blickrichtung
β	Winkel zwischen Blickrichtung zur Leinwandmitte und transformierter Blickrichtung
ΔF	Fehler zwischen erkannter und tatsächlicher Blickrichtung auf die imaginäre flache Leinwand

Literatur

- [1] Eyeware. *Eye Tracking verstehen und wie es für Sie funktionieren kann: Definitionen, Metriken und Anwendungen*. März 2022. URL: <https://eyeware.tech/de/blog/was-ist-eye-tracking/>.
- [2] Mark A. Mento. *Different Kinds of Eye Tracking Devices*. Mai 2025. URL: <https://www.bitbrain.com/blog/eye-tracking-devices>.
- [3] Amer Al-Rahayfeh und Miad Faezipour. „Enhanced frame rate for real-time eye tracking using circular hough transform“. In: *2013 IEEE Long Island Systems, Applications and Technology Conference (LISAT)*. Farmingdale, NY, USA: IEEE, Mai 2013, S. 1–6. DOI: [10.1109/lisat.2013.6578214](https://doi.org/10.1109/lisat.2013.6578214).
- [4] Ying Yu. „Technology Development and Application of IR Camera: Current Status and Challenges“. In: *Infrared and Millimeter Wave* 1.1 (2023). ISSN: 2516-371X. URL: http://166.62.7.99/assets/default/article/2023/04/27/article_1682608990.pdf.
- [5] et al. Martin Boehme. „Remote Eye Tracking: State of the Art and Directions for Future Development“. In: *COGAIN 2006 'Gazing into the Future'*. 2006. URL: <https://www.inb.uni-luebeck.de/fileadmin/files/publications/inb-publications/pdfs/BoMeMaBa06.pdf>.
- [6] Asadullah Dal Alireza Ghaderi. *Python-Gaze-Face-Tracker*. Nov. 2023. URL: <https://github.com/alireza787b/Python-Gaze-Face-Tracker>.
- [7] et al. Antoine Lamé. *GazeTracking*. Juli 2024. URL: <https://github.com/antoinelame/GazeTracking>.
- [8] Eyeware Tech SA. *Beam Eye Tracker API Code example*. 2021. URL: https://docs.beam.eyeware.tech/getting_started.html.
- [9] Eyeware Tech SA. *Beam Eye Tracker*. 2024. URL: <https://beam.eyeware.tech/>.
- [10] Canva. *Canva*. 2025. URL: <https://www.canva.com/>.
- [11] Eyeware Tech SA. *Beam SDK documentation*. 2021. URL: https://docs.beam.eyeware.tech/getting_started.html.
- [12] Eraser Labs Incorporate. *Eraser Git Diagrammer*. 2025. URL: <https://carla.readthedocs.io/en/stable/>.
- [13] Carla Team. *Carla Documentation*. Englisch. 2025. URL: <https://carla.readthedocs.io/en/stable/>.

- [14] Carla Team. *Carla Client API Code example*. URL: https://carla.readthedocs.io/en/0.9.2/python_api_tutorial/.
- [15] Carla Team. *Carla Open-source simulator for autonomous driving research*. 2025. URL: <https://carla.org/>.
- [16] et al. Dr. Frank Antwerpes Bijan Fink. *Gesichtsfeld*. Deutsch. Juli 2019. URL: <https://flexikon.doccheck.com/de/Gesichtsfeld#:~:text=In%20der%20anatomischen%20Ruhelage%20hat%2C%20von%20200%20x%20135%20Grad..>
- [17] MathWorks. *MATLAB*. Dez. 2024.
- [18] Markus Hohenwarter et al. *GeoGebra*. Apr. 2025.